



**Institutul Multimedia**  
româno - elvețian

**SUPORT DE CURS**

# **Baze de date si MySQL**

**Modulul 4 – Tehnologii Web avansate**

**Specializare:**



**WebProgramator**

## CUPRINS

|                                                                                                 |    |
|-------------------------------------------------------------------------------------------------|----|
| CAPITOLUL 1. CONCEPTE DE BAZA .....                                                             | 3  |
| 1.1. Notiunile de data si informatie .....                                                      | 3  |
| CAPITOLUL 2. CONCEPTUL DE BAZA DE DATE (BD) SI SISTEM DE GESTIUNE A BAZELOR DE DATE (SGBD)..... | 4  |
| 2.1. Evolutia si caracteristicile bazelor de date.....                                          | 4  |
| 2.2. Utilitatea si avantajele bazelor de date .....                                             | 5  |
| 2.3. Independenta datelor .....                                                                 | 5  |
| 2.4. Arhitectura unei baze de date.....                                                         | 6  |
| 2.5. Sisteme de gestiune a bazelor de date (SGBD) .....                                         | 8  |
| CAPITOLUL 3. MODELE DE REPREZENTARE A BAZELOR DE DATE .....                                     | 10 |
| 3.1. Modelul ierarhic.....                                                                      | 10 |
| 3.2. Modelul retea.....                                                                         | 10 |
| 3.3. Modelul relational.....                                                                    | 10 |
| 3.4. Modelul obiectual .....                                                                    | 11 |
| CAPITOLUL 4. TIPURI DE DATE SI TIPURI DE COLOANE UTILIZATE DE MYSQL .....                       | 13 |
| 4.1. Tipuri de date .....                                                                       | 13 |
| 4.2. Tipuri de coloane.....                                                                     | 14 |
| CAPITOLUL 5. OPERATORI SI FUNCTII .....                                                         | 19 |
| 5.1. Operatori MySQL .....                                                                      | 19 |
| 5.2. Functii MySQL .....                                                                        | 22 |
| CAPITOLUL 6. CREAREA, MODIFICAREA SI STERGerea BAZELOR DE DATE SI A TABELELOR.....              | 30 |
| 6.1. Crearea si selectarea bazelor de date.....                                                 | 30 |
| 6.2. Crearea tabelelor .....                                                                    | 31 |
| 6.3. Indexarea tabelelor.....                                                                   | 33 |
| 6.4. Modificarea structurii tabelelor .....                                                     | 34 |
| 6.5. Stergerea tabelelor si a bazelor de date .....                                             | 35 |
| CAPITOLUL 7. REGASIREA INFORMATIILOR IN TABELE .....                                            | 37 |
| 7.1. Instructiunea SELECT.....                                                                  | 37 |
| 7.2. Clauza WHERE.....                                                                          | 38 |
| 7.3. Cautarea prin comparatie (LIKE).....                                                       | 38 |
| 7.4. Limitarea rezultatelor (LIMIT) .....                                                       | 39 |
| 7.5. Gruparea randurilor (GROUP BY).....                                                        | 39 |
| 7.6. Ordonarea rezultatelor (ORDER BY).....                                                     | 39 |
| 7.7. Numararea randurilor (COUNT).....                                                          | 39 |
| 7.8. Selectarea inregistrarilor din mai multe tabele (JOIN).....                                | 39 |
| CAPITOLUL 8. INSTRUCIUNI DE BAZA .....                                                          | 41 |
| 8.1. Introducerea inregistrarilor.....                                                          | 41 |
| 8.2. Actualizarea inregistrarilor - instructiunea UPDATE .....                                  | 42 |
| 8.3. Instructiunea REPLACE .....                                                                | 43 |
| 8.4. Stergerea inregistrarilor - instructiunea DELETE .....                                     | 43 |

## Capitolul 1. CONCEPTE DE BAZA

Informatia inseamna putere, indiferent de domeniul in care activam. A gestiona informatia inseamna a dispune de putere. Gestionarea consta in stocarea, prelucrarea si vanzarea informatiei. Informatia consta in cifre, texte, imagini, sunete, miscare.

Aparitia si dezvoltarea tehnicii de calcul a facilitat transpunerea pe calculator a tuturor tehnicilor manuale de stocare a datelor dezvoltate de-a lungul timpului, si de asemenea a permis organizarea si gestionarea acestora indiferent de domeniul in care se aplica: economie, stiinta, social sau cultural.

Prin intermediul sistemelor de gestiune a bazelor de date (SGBD) se asigura accesul automat, rapid si sigur la volume mari de date care pot fi chiar si dispersate. Aceste sisteme de gestiune consitutie un element esential, primind o importanta foarte mare in societatea noastra.

### 1.1. Notiunile de data si informatie

In cadrul activitatii noastre, suntem confruntati permanent cu necesitatea de a lua decizii. Luarea deciziilor este strans legata de existenta informatiilor legate de situatia asupra careia trebuie sa decidem. Astfel, o decizie trebuie sa fie bazata pe informatii clare, complete si actuale.

Nootiunile de date si informatii sunt adesea considerate de unii echivalente, insa trebuie facuta distinctie clara intre acesti termeni.

*Datele* sunt fapte din lumea reala bazate pe observatii si masuratori. *Informatiile* sunt rezultatul interpretarii datelor de catre un anumit subiect si confera acestuia capacitatea de a lua decizii. Asadar datele devin informatii in momentul interactiunii lor cu un sistem capabil sa le interpreteze.

Datele pot fi interpretate in mod diferit de catre subiecti, ceea ce conduce la rezultate diferite. Datele au caracter obiectiv, in timp ce informatiile sunt pur subiective.

Sistemele de calcul prelucreaza date, nu informatii. Astfel, putem vorbi despre sisteme de prelucrare a datelor si despre baze de date, si nu de informatii.

## Capitolul 2. CONCEPTUL DE BAZA DE DATE (BD) SI SISTEM DE GESTIUNE A BAZELOR DE DATE (SGBD)

Informatiile trebuie sa fie disponibile in timp util, sa fie corecte, sa nu fie contradictorii si redundante. Aceste cerinte pot fi puse in valoare doar printr-un proces complex realizabil prin intermediul unor pasi ce cuprind culegerea, memorarea, organizarea, regasirea si prelucrarea datelor in vederea utilizarii lor ca informatii.

### 2.1. Evolutia si caracteristicile bazelor de date

Organizarea datelor pe un anumit suport de memorie a parcurs, in timp, mai multe etape, intre care putem aminti: fisierele, fisierele cu legaturi si in cele din urma bazele de date actuale.

*Prima etapa* in evolutia organizarii si prelucrarii datelor o reprezinta fisierul. Un fisier consta intr-un ansamblu de inregistrari fizice. In aceasta etapa datele sunt organizate sub forma de fisiere secventiale stocate pe benzi magnetice (ca suport de memorare). Structura logica si structura fizica sunt similare, neexistand o diferenta clara intre acestea. Modul de proiectare a datelor este realizat in scopul de a deservi in principal unei singure aplicatii, aceasta fiind dependenta de date si ca urmare, in cazul aparitiei unor modificari in structura datelor, acest fapt determina modificari si asupra programului (aplicatiei).

*A doua etapa* este marcata de separarea structurii logice de cea fizica, posibila prin independenta fizica a datelor. Sunt utilizate fisiere secvential-indexate, si cu acces direct, avand ca suport de memorare discul magnetic. Separarea celor doua structuri asigura independenta aplicatiilor atat fata de echipamentele hardware cat si fata de modul de organizare a fisieleror. Aceasta inseamna ca se pot schimba dispozitivele de memorare fara a afecta aplicatiile, iar aceasta schimbare afecteaza, eventual, doar structura fizica a datelor, nu si structura logica. In aceasta etapa apar primele facilitati simple de protectie a datelor.

*Etapa a treia* marcheaza aparitia fisieleror integrate. In acest sens sunt rezolvate o serie de probleme legate de redondanta datelor (aceleasi date folosite in aplicatii diferite sunt inregistrate in mai multe fisiere), de absenta unor legaturi logice intre datele din diferite fisiere si de flexibilitatea redua (aparitia unei noi aplicatii necesita fie crearea de fisiere noi, fie reorganizarea celor existente). Organizarea datelor sub forma fisieleror integrate reduce redondanta datelor, fiind posibila utilizarea in comun a celorasi date fizice de catre mai multe aplicatii. Integrarea fisieleror determina o structura logica care sa corespunda cerintelor tuturor aplicatiilor. Aceste structuri integrate constituie originea notiunii de model conceptual din cadrul bazelor de date actuale. El contine descrierea tuturor datelor si a legaturilor dintre ele.

*Etapa a patra* este etapa bazelor de date propriu-zise. Odata cu utilizarea fisieleror integrate, ce confera mai multor aplicatii accesul la aceleasi date, a aparut si necesitatea realizarii aplicatiilor independent de structura logica a datelor. Desprinderea din cadrul aplicatiilor a descrierii structurilor de date determina crearea modelului extern. Astfel, pe langa independenta fizica a datelor se adauga

si independenta logica, in sensul ca sunt posibile modificari in structura logica a unei baze de date, fara ca acestea sa afecteze aplicatiile.

In aceasta etapa se concretizeaza modelul conceptual ca nivel virtual de organizare a datelor la care se adauga modelele externe asociate fiecarui utilizator.

Tot in aceasta etapa apar tehnici si proceduri speciale pentru protectia si securitatea datelor, si de asemenea si functia de administrare a bazei de date, iar redundanta datelor este redusa la minim.

## 2.2. Utilitatea si avantajele bazelor de date

Caracteristica principala a aplicatiilor de baze de date consta in faptul ca accentul este pus pe operatiile de memorare si regasire, care sunt efectuate asupra unui volum mare de date, si mai putin asupra operatiilor de prelucrare.

Principala operatie este cea de regasire a datelor, in scopul obtinerii de informatii din baza de date. Alaturi de informatiile de regasire apar si operatii de memorare, in scopul introducerii de noi date in baza de date, operatii de stergere sau de actualizare a unor date deja existente acolo.

Organizarea datelor in baze de date constituie o forma de centralizare a acestora si prezinta o serie de avantaje cum ar fi:

- ✓ reducerea redundantei datelor memorate;
- ✓ evitarea inconsistentei datelor memorate;
- ✓ posibilitatea partajarii datelor de catre mai multe aplicatii;
- ✓ introducerea standardelor impuse;
- ✓ posibilitatea aplicarii restrictiilor de securitate pentru fiecare tip de acces la date, pentru fiecare data si la nivel de utilizator;
- ✓ mentinerea integritatii datelor care presupune atat consistenta datelor cat si plauzibilitatea lor, prin introducerea unor proceduri de validare corespunzatoare.

## 2.3. Independenta datelor

Independenta datelor rezolva problema delimitarii intre reprezentarea fizica a datelor si imaginea pe care o are utilizatorul asupra acestor date. Acesta inseamna ca modul de memorare si organizare a datelor este transparent pentru utilizator.

Astfel, utilizatorul trebuie sa fie preocupat doar de problema pe care o are de rezolvat, detaliile de implementare nefiind esentiale pentru el, acestea ramanand in sarcina sistemului.

Independenta datelor se prezinta sub doua aspecte:

- ✓ independenta *fizica* a datelor;
- ✓ independenta *logica* a datelor.

### 2.3.1. Independenta fizica a datelor

Consta in faptul ca modificarile in structura fizica de memorare a datelor nu trebuie sa afecteze aplicatia si reciproc, modificarile aplicatiei nu trebuie sa determine modificari ale structurii fizice a datelor.

Din punctul de vedere al aplicatiei, datele trebuie privite ca entitati cu nume, orice referire la ele din cadrul aplicatiei facandu-se prin aceste nume. Ramane la latitudinea sistemului si a administratorului bazei de date modul optim de organizare a datelor si politicile de acces.

### 2.3.2. Independenta logica a datelor

Se refera la posibilitatea de a face modificari in structura logica a bazei de date; este strans legata de problema adaugarii de noi entitati logice (fie ele campuri sau inregistrari) la structura bazei de date si de modificarea relatiilor existente intre ele. In acest sens este posibila dezvoltarea bazei de date prin definirea de noi campuri sau inregistrari si adaugarea de date.

Singura problema nerezolvabila este eliminarea de entitati din baza de date, aceasta avand repercursiuni asupra utilizatorilor care fac referire la acea entitate eliminata.

Prin independenta logica a datelor se urmareste ca pentru fiecare utilizator se sa creeze impresia ca el este singurul beneficiar al unor date pe care, in realitate, le foloseste in comun cu alti utilizatori.

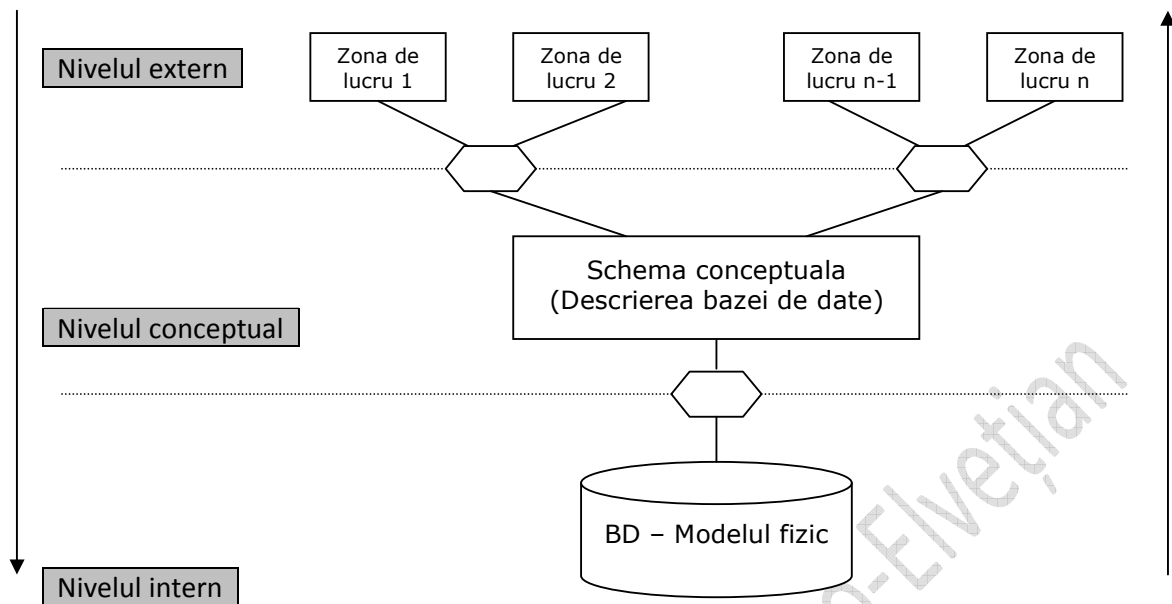
Independenta logica a datelor este mult mai greu de realizat decat cea fizica, depinzand in mare masura de modelul de date folosit (ierarhic, retea, relational).

## 2.4. Arhitectura unei baze de date

Asigurarea independentei fizice si logice a datelor impune adoptarea unor arhitecturi de baze de date organizate pe cel putin trei nivele:

- ✓ nivelul intern (baza de date fizica)
- ✓ nivelul conceptual (modelul conceptual, schema conceptuala)
- ✓ nivelul extern (modelul extern, subschema, imaginea bazei de date)

Schema generala a unei baze de date care respecta o astfel de organizare este prezentata in figura de mai jos:



### 2.4.1. Nivelul intern

Poarta numele si de baza de date fizica, fiind format dintr-o colectie de fisiere care contin datele fizice, la care se adauga diverse structuri auxiliare menite sa asigure accesul operativ la aceste date. Structurile auxiliare pot fi: directoare, indecsi, pointeri, etc. Baza de date fizica este stocata in general pe discuri magnetice sau optice, iar modul de organizare este influentat in mare masura de configuratia echipamentelor hardware si de sistemul de operare.

Schimbarea sistemului de operare sau modificari in configuratia echipamentelor hardware pot atrage modificari ale bazei de date fizice. Daca este satisfacuta conditia de independenta fizica a datelor, aceste modificari nu ar trebui sa afecteze nivelele superioare nivelului intern.

### 2.4.2. Nivelul conceptual

Este o abstractizare, descrie structura logica a datelor din baza de date. Fiecare baza de date are un model conceptual propriu prin care sunt numite si descrise toate unitatile logice din baza de date, impreuna cu legaturile dintre acestea.

Modelul conceptual specifica ce anume poate face parte din baza de date, respectiv ceea ce nu poate face parte. Acest lucru este realizabil prin specificarea unor constrangeri explicite. Constrangerile se refera la restrictii asupra valorilor pe care le pot lua datele si sunt declarate in descrierea modelului conceptual. Daca o operatie de actualizare conduce la violarea vreunei constrangeri, atunci executia acesteia este oprita.

Prin modelul conceptual este realizata independenta fizica a datelor. Modelului conceptual i se asociaza o transformare care defineste modul in care structura logica de date este transpusa in structura fizica de memorare.

### 2.4.3. Nivelul extern

Poate fi privit ca o viziune a unui utilizator sau grup de utilizatori asupra bazei de date. Un model extern contine o parte a unitatilor logice dintr-un model conceptual, dar poate contine si unitati logice care nu exista in modelul conceptual si care nu au corespondent direct in baza de date fizica. Acest tip de unitati logice se numesc virtuale si pot fi obtinute prin modificarea unor unitati logice reale sau prin combinarea acestora.

Modelul extern este cel mai apropiat de utilizator, fiecare avand posibilitatea de a-l individualiza in raport cu cerintele sale specifice. Termenul tehnic folosit pentru modelul extern este acela de vedere. Utilizarea acestora asigura securitatea bazei de date prin limitarea accesului la date a anumitor categorii de utilizatori, pe de alta parte un utilizator poate avea drepturi de acces diferite in cadrul mai multor vederi.

O alta functie a vederilor este aceea ca fiind individualizate si oferind o forma simplificata asupra bazei de date, ele ascund utilizatorului acele parti din descrierea bazei de date care nu-l intereseaza pe acesta.

Prin modelul extern se realizeaza independenta logica a datelor. Fiecare vedere are ca si corespondent a descriere a unitatilor logice din modelul conceptual. Aceasta descriere defineste transformarile prin care structura logica de la nivelul conceptual devine model extern.

## 2.5. Sisteme de gestiune a bazelor de date (SGBD)

Un SGBD se constituie ca un intreg ansamblu software care trateaza toate cererile de acces ale utilizatorilor la baza de date. El actioneaza ca o interfata care permite crearea, actualizarea si consultarea bazei de date.

O cerere de acces la baza de date este formulata de catre utilizator, este interceptata de catre SGBD si interpretata de catre o componenta a acestuia (LMD – limbaj de manipulare a datelor), rezultand o reprezentare in format intern a interogarii. In continuare interogarea parcurge o serie de etape succesive de prelucrare, rezulta o serie de comenzi de acces la fisierele fizice din baza de date. Cererile de acces la fisierele fizice, rezultate din transformarea interogarii, sunt prelucrate si rezolvate de catre un sistem de gestiune al fisierelor. Datele extrase din fisierele fizice, sub forma unor siruri de biti, parcurg apoi calea inversa, rezultatul fiind un raspuns formulat sub forma de tabele, rapoarte, etc.

Principalele functii ale unui SGBD sunt:

- ✓ descrierea datelor,
- ✓ actualizarea si interogarea bazei de date,
- ✓ conversia datelor dintr-un format intr-altul,
- ✓ asigurarea controlului integritatii, concurentei si securitatii datelor.

### Descrierea datelor

SGBD-ul trebuie sa fie capabil sa defineasca datele prin intermediul unui limbaj specializat DDL (Description Data Language) intr-o forma unificata numita *schema sursa* si apoi sa compileze aceasta schema intr-o forma interna numita *schema obiect*.

Cele 3 niveluri de descriere a datelor (extern, conceptual si intern) si procedurile de trecere dintr-unul in altul sunt inregistrate in *dictionarul de date*, unde informatiile sunt memorate in format sursa si obiect (compilat). Un dictionar de date organizat sub forma de baza de date se numeste *metabaza*.

### Actualizarea si interogarea bazei de date

In SGBD-uri implementarea functiei de interogare se realizeaza prin intermediul unor limbaje declarative, care permit cautarea datelor dupa continut, fara a preciza procedurile de acces. cel mai utilizat limbaj de interogare a bazelor de date este in momentul de fata SQL.

O interogare de deruleaza in 4 etape: formularea interogarii utilizand limbajul de manipulare a datelor (DML), compilare, optimizare si apoi executie.

SGBD-urile poseda un compilator (analizor) de cereri de interogare, care permite analiza sintactica si evaluarea iterogarii. Pentru validarea interogarii este folosit dictionarul de date. Interogariile sunt descompuse in operatori relationali, pe care ii structureaza sub forma de arbore, in care nodurile reprezinta operatorii iar elementele de pe ultimul nivel reprezinta relatiile.

O cerere de interogare este apoi optimizata in vederea executiei. Dupa procesul de optimizare, cererile sunt transformate in tranzactii care vor fi executate, avandu-se in vedere doua aspecte: gestiunea tranzactiilor concurente si executia propriu-zisa.

### Conversia datelor

Consta in posibilitatile pe care SGBD-ul le ofera pentru trecerea datelor intre cele 3 niveluri: extern, conceptual si intern. Pentru aceasta SGBD-ul trebuie sa cunoasca corespondentele existente intre niveluri.

### Controlul integritatii datelor

SGBD-ul trebuie sa asigure coerenta datelor. Toate regulile implicite sau explicite care asigura integritatea datelor se numesc restrictii de integritate. Dintre acestea amintim: integritatea cheii primare, integritatea referirii si integritatea de domeniu.

### Securitatea datelor

Un SGBD trebuie sa garanteze securitatea datelor. Acest lucru se realizeaza printr-un mecanism de control al drepturilor de acces la date si de restaurare a bazei de date in caz de erori.

## Capitolul 3. MODELE DE REPREZENTARE A BAZELOR DE DATE

Asa cum am mai spus, datele in sine nu constituie informatie. Pentru a putea fi utile, ele trebuie organizate intr-un anumit mod. Modelarea datelor urmareste exact acest mod de a organiza datele astfel incat ele sa reprezinte cat mai fidel situatia reala si sa fie adaptate reprezentarii si prelucrarii electronice.

### 3.1. Modelul ierarhic

A fost primul model utilizat in bazele de date, avand la baza structura arborescenta, in care nodul parinte poate avea mai multe noduri copil, in timp ce un nod copil poate avea doar un singur nod parinte. Acest model se bazeaza pe relatii de tip parinte-copil.

Modelul de date ierarhic foloseste doua forme de structurare a datelor: tipurile de inregistrari (pentru reprezentarea tipurilor de entitati) si legaturile (pentru reprezentarea relatiilor intre multi de entitati).

### 3.2. Modelul retea

Modelul retea are la baza structura de tip retea. Acest model este similar cu modelul ierarhic, un nod parinte putand avea mai multi copii, deosebirea constand in faptul ca un nod copil poate avea mai multi parinti.

Reprezentarea modelului de date retea se realizeaza tot prin intermediul diagramelor de structura, care se aseamana cu un graf, nodurile fiind inlocuite de tipurile de inregistrare.

### 3.3. Modelul relational

Este modelul conceptual cel mai des folosit de SGBD-uri. Sistemul care permite gestionarea bazelor de date relationale se numeste sistem de gestiune a bazelor de date relationale – SGBDR.

O baza de date relationala este alcatuita din mai multe tabele intre care se pot stabili relatii. Un tabel contine inregistrari (randuri). O inregistrare este alcatuita din mai multe campuri (coloane sau attribute).

In teoria relationala, intr-un tabel, inregistrarile nu se pot repeta. Liniile sunt identificate prin valorile atributelor care le compun.

O combinatie de attribute care permite identificarea inregistrarilor poarta denumirea de cheie candidata. O cheie candidata trebuie sa respecte trei conditii: sa fie unica (unicitatea se refera la faptul ca o cheie identifica in mod unic o inregistrare), ca nu aiba valori nule (attributele care compun cheia trebuie sa aiba valori nenule) si sa aiba o compozitie minimala. Dintre cheile candidate se alege o cheie primara. Criteriile de alegere sunt lungimea cheii si usurinta in manipulare.

### 3.3.1. Relatii intre tabele

Relatiile intre tabele pot fi de mai multe feluri:

- ✓ relatii de tip 1 la 1 (o cheie dintr-o tabela apare o singura data intr-o alta tabela cu care aceasta se afla in relatie)
- ✓ relatii de tip 1 la n (cheile dintr-o tabela apar de mai multe ori intr-o alta tabela cu care aceasta se gaseste in relatie. Acest tip de relatie este cel mai frecvent utilizat)
- ✓ relatii de de tip n la n (creaza probleme in cadrul bazelor de date normalizate, astfel incat ele trebuie sparte in serii de relatii unul la mai multe; valoarea cheii dintr-o tabela poate sa apara de mai multe ori intr-o tabela cu care aceasta se afla in relatie, insa si reciproca este valabila, adica o cheie din cea de-a doua tabela poate, la randul ei, sa apara de mai multe ori in prima tabela )

### 3.3.2. Normalizarea bazelor de date

Procesul prin care o baza de date relationala este adusa la o forma optimizata, prin transformari succesive pe baza unor reguli, se numeste *normalizare*.

Seturile de reguli, prin care o baza de date este adusa in fiecare etapa intr-o noua forma standard, se numesc *forme normale (FN)*. Exista 5 forme normale. Primele doua forme normale (FN1 si FN2) au fost definite de Codd. Apoi, Codd si Boyce au elaborat FN3, iar Fagin a elaborat FN4 si FN5.

Prima forma normala elimina campurile compuse si cele repetitive. Regulile sunt:

- ✓ eliminarea informatiei redundante;
- ✓ crearea de tabele separate pentru datele aflate in relatie.

A doua forma normala elimina dependentele functionale partiale. Regula este ca toate campurile unui tabel sa depinda doar de cheia primara si nu de parti (portiuni) ale acesteia.

A treia forma normala elimina dependentele functionale tranzitive. Regula este ca nici un atribut nu depinde de un alt atribut care nu este cheia primara.

A patra forma normala elimina dependentele multivaloare suplimentare (pentru un tabel este permisa numai una singura), iar a cincea forma normala elimina dependentele jonctiune.

## 3.4. Modelul obiectual

Modelul obiectual determina lucrul cu baze de date obiectuale, care au o structura ierarhica. La nivelul cel mai de sus se afla familia de clase, pe al doilea nivel ierarhic este clasa de obiecte, iar pe nivelul trei se afla obiectul.

Modelul orientat obiect permite modelarea directa a realitatii prin intermediul obiectelor definite ca entitati cu identitate proprie si caracterizate prin stare si comportament. Obiectele pot comunica intre ele prin intermediul mesajelor.

*Identitatea* unui obiect este proprietatea acestuia care îl distinge de alte obiecte și este, de regulă, o adresă logică invariantă (pointer). Astfel, spre deosebire de modelul relational, unde datele sunt identificate prin valori ale cheilor primare desemnate de utilizator, în cazul modelului obiectual de identificare a obiectelor este asigurată automat de sistem și este transparentă utilizatorului.

*Starea* unui obiect este definită de valorile atributelor sale. Un atribut este definit printr-un nume și poate lua valori elementare (numeric, alfanumeric, etc) sau complexe (structuri de valori multiple, referințe spre alte obiecte, tipuri utilizator, etc). Atributele se pot concretiza în locații de memorie în care sunt stocate datele aferente.

*Comportamentul* unui obiect este definit de setul de operații (metode) aplicabile obiectului respectiv. Operațiile unui obiect constituie modalități de răspuns la mesaje primite din exterior. Operațiile sunt de regulă proceduri și funcții care acționează asupra obiectului respectiv.

*Abstractizarea* este procesul prin care obiectele cu aceleași atribute și cu comportament comun sunt grupate în tipuri abstracte de date numite și clase. Obiectele devin astfel instanțieri ale claselor.

*Incapsularea* constă în capacitatea obiectelor (claselor) de a conține la un loc atât date cât și operații dintre care numai o parte sunt vizibile din exterior. Astfel obiectele apar ca niște cutii negre care ascund detaliile de implementare.

*Mostenirea* este procesul prin care toate atributele și metodele publice (vizibile din exterior) ale unei clase de bază sunt preluate automat de o altă clasă înrudită, numită clasă derivată.

*Polimorfismul* semnifică posibilitatea unui obiect, instanță a unei clase, să răspundă diferit la primirea aceluiași mesaj. Polimorfismul poate fi asigurat prin redefinirea (rescrierea) metodelor moștenite în clasele derivate sau prin supraîncărcarea unei metode în cadrul aceleiași clase.

## Capitolul 4. TIPURI DE DATE SI COLOANE UTILIZATE DE MySQL

### 4.1. Tipuri de date

Sistemul MySQL foloseste mai multe tipuri de date, si anume: tipul numeric, tipul sir de caractere si tipul data si ora.

#### 4.1.1. Date numerice

Numerele pot fi:

- *intregi* : scrise in format zecimal sau in format hexazecimal;
- *in virgula mobila*

In sistemul MySQL se poate folosi si notatie stiintifica.

#### 4.1.2. Date calendaristice

Datele se specifica conform standardului SQL ANSI92, in ordinea *an-luna-zi*, iar ora in format *ora-minute-secunde*. Data si ora pot fi reprezentate si impreuna.

#### 4.1.3. Siruri de caractere

Este considerat sir de caractere orice grup de caractere incadrate intre ghilimele simple (') sau duble ("). Sirurile de caractere pot contine si anumite caractere speciale, pentru afisarea carora se folosesc secventele escape. O secventa escape incepe cu caracterul backslash (\) si continua cu caracterul special care trebuie introdus in sir.

Secventele escape folosite in sirurile MySQL sunt:

| Secventa escape | Semnificatie                                                                |
|-----------------|-----------------------------------------------------------------------------|
| \n              | Caracterul linie noua                                                       |
| \r              | Caracterul inceput de linie (retur de car – carriage return, CR)            |
| \t              | Caracterul TAB                                                              |
| \b              | Caracterul BACKSPACE                                                        |
| \z              | Caracterul ASCII 26 (CTRL+Z)                                                |
| \\              | Caracterul <i>backslash</i>                                                 |
| \"              | Caracterul ghilimele duble                                                  |
| \'              | Caracterul ghilimele simple                                                 |
| \0              | Caracterul ASCII 0 (octet NULL)                                             |
| \%              | Caracterul %, folosit pentru cautarea caracterului % in siruri de caractere |
| \_              | Caracterul _, folosit pentru cautarea caracterului _ in siruri de caractere |

## 4.2. Tipuri de coloane

Un tabel este alcatuit din una sau mai multe coloane. La crearea unui tabel trebuie specificat tipul fiecărei coloane. MySQL folosește mai multe tipuri de date, ce pot fi grupate în trei categorii:

- ✓ numerice
- ✓ calendaristice
- ✓ siruri de caractere

### 4.2.1. Coloane de tip numeric

MySQL furnizează tipuri numerice pentru valori întregi și cu virgula mobilă. Tipurile pot fi selectate în funcție de domeniul de valori care trebuie reprezentat.

Tipurile de date numerice folosite în MySQL sunt următoarele:

- ✓ **TINYINT**: număr întreg de foarte mici dimensiuni, cuprins între -128 și 127, sau între 0 și 255 dacă este precizat atributul **UNSIGNED**. Mai permite și atributele **ZEROFILL** (completare cu zerouri ne semnificative, scrise în stanga numărului) și **AUTO\_INCREMENT**. Are valoarea prestabilită **NULL** dacă în coloană se acceptă valoarea **NULL**, sau 0 în cazul în care coloana are specificat atributul **NOT NULL**. Necesită un spațiu de stocare de un octet.
- ✓ **SMALLINT**: număr întreg de dimensiuni mici, cuprins între -32.768 și 32.767, sau între 0 și 65.535 dacă este precizat atributul **UNSIGNED**. Mai permite și atributele **ZEROFILL** și **AUTO\_INCREMENT**. Are valoarea prestabilită **NULL** dacă în coloană se acceptă valoarea **NULL**, sau 0 în cazul în care coloana are specificat atributul **NOT NULL**. Necesită un spațiu de stocare de 2 octeți.
- ✓ **MEDIUMINT**: număr întreg de dimensiuni medii, cuprins între -8.388.608 și 8.388.607, sau între 0 și 16.777.215 dacă este precizat atributul **UNSIGNED**. Mai permite și atributele **ZEROFILL** și **AUTO\_INCREMENT**. Are valoarea prestabilită **NULL** dacă în coloană se acceptă valoarea **NULL**, sau 0 în cazul în care coloana are specificat atributul **NOT NULL**. Necesită un spațiu de stocare de 3 octeți.
- ✓ **INT**: număr întreg de dimensiuni normale, cuprins între -2.147.483.648 și 2.147.483.647, sau între 0 și 4.294.967.295 dacă este precizat atributul **UNSIGNED**. Mai permite și atributele **ZEROFILL** și **AUTO\_INCREMENT**. Are valoarea prestabilită **NULL** dacă în coloană se acceptă valoarea **NULL**, sau 0 în cazul în care coloana are specificat atributul **NOT NULL**. Necesită un spațiu de stocare de 4 octeți.
- ✓ **INTEGER**: sinonim cu **INT**.
- ✓ **BIGINT**: număr întreg de dimensiuni mari, cuprins între  $-2^{63}$  și  $2^{63}-1$  (-9.223.372.036.854.775.808 și 9.223.372.036.854.775.807), sau între 0 și  $2^{64}-1$  (18.446.744.073.709.551.615) dacă este precizat atributul **UNSIGNED**. Mai permite și

atributele ZEROFILL si AUTO\_INCREMENT. Are valoarea prestabilita NULL daca in coloana se accepta valoarea NULL, sau 0 in cazul in care coloana are specificat atributul NOT NULL. Necesita un spatiu de stocare de 8 octeti.

- ✓ FLOAT: numar mic, cu virgula mobila, in simpla precizie (mai putin precis decat DOUBLE); permite atributul ZEROFILL; valorile minime diferite de zero sunt  $\pm 1.175494351E-38$  si valorile maxime diferite de zero sunt  $\pm 3.402823466E+38$ . Are valoarea prestabilita NULL daca in coloana se accepta valoarea NULL, sau 0 in cazul in care coloana are specificat atributul NOT NULL. Necesita un spatiu de stocare de 4 octeti.
- ✓ DOUBLE: numar mare, cu virgula mobila, in dubla precizie (mai precis decat FLOAT); permite atributul ZEROFILL; valorile minime diferite de zero sunt  $\pm 2.2250738585072014E-308$  si valorile maxime diferite de zero sunt  $\pm 1.7976931348623157E+308$ . Are valoarea prestabilita NULL daca in coloana se accepta valoarea NULL, sau 0 in cazul in care coloana are specificat atributul NOT NULL. Necesita un spatiu de stocare de 8 octeti.
- ✓ REAL: sinonim cu DOUBLE
- ✓ DECIMAL(M,D): numar cu virgula mobila, stocat sub forma unui sir (1 octet pentru fiecare cifra, virgula sau caracter '-'); permite atributul ZEROFILL; intervalul maxim de valori este acelasi ca si pentru DOUBLE, insa domeniul efectiv este determinat de  $M$  si  $D$ .
- ✓ Daca  $D$  este 0 atunci valorile nu au cifre dupa virgula sau parte fractionara.
- ✓ Are valoarea prestabilita NULL daca in coloana se accepta valoarea NULL, sau 0 in cazul in care coloana are specificat atributul NOT NULL. Necesita un spatiu de stocare de  $M$  octeti (sau  $M+2$ , depinde de versiunea de MySQL).
- ✓ NUMERIC(M,D): sinonim cu DECIMAL(M,D)

#### 4.2.2. Coloane de tip data si ora

MySQL furnizeaza tipuri pentru reprezentarea datelor de tip data calendaristica si a orelor in diferite forme. Aceste tipuri de date sunt flexibile din punct de vedere al introducerii, pot exista tipuri de data si ora fie impreuna, fie in combinatie.

MySQL verifica doar doua elemente de validitate: luna sa fie cuprinsa intre 0 si 12 si ziua sa fie intre 0 si 31. MySQL nu verifica automat daca a 30-a zi a lunii februarie este o data valida. Responsabilitatea de verificare a datelor cade in sarcina programatorului.

Tipurile data si ora utilizate de coloanele MySQL sunt urmatoarele:

- ✓ DATE: data in format "AAAA-LL-ZZ". Domeniul in care poate fi precizata o valoare de tip data este intre "1000-01-01" si "9999-12-31". Valoarea zero este: "0000-00-00", iar valoarea prestabilita este NULL daca in coloana se accepta valoarea NULL, sau "0000-00-00" daca coloana are atributul NOT NULL. Necesita spatiu de stocare de 3 octeti.

- ✓ **TIME:** valoare de tip ora in format "hh:mm:ss" (sau "-hh:mm:ss" pentru valori negative); reprezinta timpul scurs, dar poate fi tratat si ca ora din zi. Domeniul in care poate fi precizata o valoare de tip TIME este cuprins intre "-838:59:59" si "838:59:59". Valoarea zero este: "00:00:00", iar valoarea prestabilita este NULL daca coloana accepta valoarea NULL sau "00:00:00" in cazul in care coloana are atributul NOT NULL. Necesita spatiu de stocare de 3 octeti.
- ✓ **DATETIME:** combinatie intre data si ora (ambele componente fiind obligatorii), stocata in format "AAAA-LL-ZZ hh:mm:ss". Domeniul in care poate fi precizata o valoare de tip DATETIME este cuprins intre "1000-01-01 00:00:00" si "9999-12-31 23:59:59". Valoarea zero este: "0000-00-00 00:00:00", iar valoarea prestabilita este NULL daca coloana accepta valoarea NULL sau "0000-00-00 00:00:00" in cazul in care coloana are atributul NOT NULL. Necesita spatiu de stocare de 8 octeti.
- ✓ **TIMESTAMP[(M)]:** *amprenta de timp* (data si ora). Formatul de afisare este "AAAALLZZhhmmss", insa depinde de valoarea lui *M*. In functie de aceste valori formatul de afisarea este conform tabelului urmator:

| Valoare M    | Format de afisare |
|--------------|-------------------|
| 14 sau lipsa | AAAALLZZhhmmss    |
| 12           | AALLZZhhmmss      |
| 10           | AALLZZhhmm        |
| 8            | AAAALLZZ          |
| 6            | AALLZZ            |
| 4            | AALL              |
| 2            | AA                |

Coloanele de tip **TIMESTAMP** sunt deseori utilizate pentru data si ora la care a fost creata sau modificata o inregistrare, deoarece valorile unei astfel de coloane sunt automat actualizate la fiecare operatie INSERT sau UPDATE. Necesita spatiu de stocare de 4 octeti.

- ✓ **YEAR:** an in format implicit "AAAA". Spatiul de afisare poate fi de 4 sau de 2 caractere. Valorile permise sunt intre 1901 si 2155, daca se utilizeaza spatiul de afisarea 4, si intre 1970 si 2069 daca se utilizeaza spatiul de afisare 2. Valoarea zero este "0000", iar cea prestabilita este NULL daca coloana accepta valoarea NULL si 0000 in cazul in care coloana are atributul NOT NULL. Necesita spatiu de stocare de un octet.

#### 4.2.3. Coloane de tip sir de caractere

- ✓ **CHAR(M):** sir de caractere de lungime fixa care poate ajunge la *M* caractere, *M* fiind un intreg cuprins intre 1 si 255. Sirurile mai scurte de *M* caractere sunt completate cu spatii finale, iar cele mai lungi de *M* caractere sunt trunchiate la lungimea *M*. Permite atributul BINARY. Are valoarea prestabilita NULL daca in coloana se accepta valoarea NULL, sau " " in cazul in care coloana are specificat atributul NOT NULL. Necesita un spatiu de stocare de *M* octeti.

- ✓ **VARCHAR(M)**: sir de caractere de lungime variabila, unde lungimea se poate situa intre 0 si  $M$  octeti,  $M$  fiind un intreg cuprins intre 1 si 255. Spatiile finale din cadrul valorilor sunt eliminate la stocarea acestora, iar cele mai lungi de  $M$  caractere sunt trunchiate la lungimea  $M$ . Permite atributul **BINARY**. Are valoarea prestabilita **NULL** daca in coloana se accepta valoarea **NULL**, sau " " in cazul in care coloana are specificat atributul **NOT NULL**. Necesita un spatiu de stocare de  $M$  octeti, plus un octet pentru inregistrarea lungimii. Acest tip este indicat atunci cand se stocheaza siruri a caror lungime este variabila.
- ✓ **BLOB**: valoare *BLOB (binary large object)* de dimensiuni normale. Poate stoca aproape orice obiect (fisier) de o dimensiune aproape oricât de mare (maxim  $2^{16}-1$  octeti). Are valoarea prestabilita **NULL** daca in coloana se accepta valoarea **NULL**, sau " " in cazul in care coloana are specificat atributul **NOT NULL**. Necesita un spatiu de stocare egal cu lungimea valorii, plus 2 octeti pentru inregistrarea lungimii. In cazul comparatiilor este sensibil la diferenta intre majuscule si minuscule.
- ✓ **TINYBLOB**: valoare *BLOB* de dimensiuni mici. Lungimea permisa este intre 0 si  $2^8-1$  octeti. Are valoarea prestabilita **NULL** daca in coloana se accepta valoarea **NULL**, sau " " in cazul in care coloana are specificat atributul **NOT NULL**. Necesita un spatiu de stocare egal cu lungimea valorii, plus 1 octet pentru inregistrarea lungimii. In cazul comparatiilor este sensibil la diferenta intre majuscule si minuscule.
- ✓ **MEDIUMBLOB**: valoare *BLOB* de dimensiuni medii. Lungimea permisa este intre 0 si  $2^{24}-1$  octeti. Are valoarea prestabilita **NULL** daca in coloana se accepta valoarea **NULL**, sau " " in cazul in care coloana are specificat atributul **NOT NULL**. Necesita un spatiu de stocare egal cu lungimea valorii, plus 3 octeti pentru inregistrarea lungimii. In cazul comparatiilor este sensibil la diferenta intre majuscule si minuscule.
- ✓ **LOB**: valoare *BLOB* de dimensiuni mari. Lungimea permisa este intre 0 si  $2^{32}-1$  octeti. Are valoarea prestabilita **NULL** daca in coloana se accepta valoarea **NULL**, sau " " in cazul in care coloana are specificat atributul **NOT NULL**. Necesita un spatiu de stocare egal cu lungimea valorii, plus 4 octeti pentru inregistrarea lungimii. In cazul comparatiilor este sensibil la diferenta intre majuscule si minuscule.
- ✓ **TEXT**: valoare *TEXT* de dimensiuni normale. Lungimea permisa este intre 0 si  $2^{16}-1$  octeti. Are valoarea prestabilita **NULL** daca in coloana se accepta valoarea **NULL**, sau " " in cazul in care coloana are specificat atributul **NOT NULL**. Necesita un spatiu de stocare egal cu lungimea valorii, plus 2 octeti pentru inregistrarea lungimii. In cazul comparatiilor este insensibil la diferenta intre majuscule si minuscule.
- ✓ **TINYTEXT**: valoare *TEXT* de mici dimensiuni. Lungimea permisa este intre 0 si  $2^8-1$  octeti. Are valoarea prestabilita **NULL** daca in coloana se accepta valoarea **NULL**, sau " " in cazul in care coloana are specificat atributul **NOT NULL**. Necesita un spatiu de stocare egal cu lungimea valorii, plus un octet pentru inregistrarea lungimii. In cazul comparatiilor este insensibil la diferenta intre majuscule si minuscule.

- ✓ MEDIUMTEXT: valoare *TEXT* de dimensiuni medii. Lungimea permisa este între 0 și  $2^{24}-1$  octeti. Are valoarea prestabilită NULL dacă în coloana se acceptă valoarea NULL, sau " " în cazul în care coloana are specificat atributul NOT NULL. Necesită un spațiu de stocare egal cu lungimea valorii, plus 3 octeti pentru înregistrarea lungimii. În cazul comparațiilor este insensibil la diferența între majuscule și minuscule.
- ✓ LONGTEXT: valoare *TEXT* de dimensiuni mari. Lungimea permisa este între 0 și  $2^{32}-1$  octeti. Are valoarea prestabilită NULL dacă în coloana se acceptă valoarea NULL, sau " " în cazul în care coloana are specificat atributul NOT NULL. Necesită un spațiu de stocare egal cu lungimea valorii, plus 4 octeti pentru înregistrarea lungimii. În cazul comparațiilor este insensibil la diferența între majuscule și minuscule.

Coloanele de tip BLOB și TEXT pot stoca informații a căror lungime poate să difere mult de la o înregistrare la alta (imagini, sunete, documente, etc.). Marimea unui obiect BLOB sau TEXT este determinată de tipul său, însă cea mai mare valoare care poate fi transferată între client și server este determinată de cantitatea de memorie disponibilă și de marimea buffer-ului de comunicație cu programele client.

- ✓ ENUM: enumerare; valorile coloanelor pot avea exact una dintre valorile membrilor listei. Lista de valori permise este enumerată explicit în momentul în care se creează tabelul. Membrii listei sunt indexați (numerotați) secvențial, începând cu valoarea 1. Indexul 0 este asociat membrului " " (sirul vid). Numărul maxim de membri ai unei enumerări este de 65535. Are valoarea prestabilită NULL dacă în coloana se acceptă valoarea NULL, sau prima valoare din enumerare în cazul în care coloana are atributul NOT NULL. Necesită un spațiu de stocare de un octet pentru enumerări cu 1 până la 255 membri, 2 octeti pentru enumerări cu 256 până la 65535 membri. În cazul comparațiilor este insensibil la diferența între majuscule și minuscule.
- ✓ SET: set; valorile coloanelor pot avea zero sau mai multe dintre valorile membrilor listei. Are valoarea prestabilită NULL dacă în coloana se acceptă valoarea NULL, sau " " (set vid) în cazul în care coloana are atributul NOT NULL. Necesită un spațiu de stocare de un octet pentru seturi cu 1 - 8 membri, 2 octeti pentru seturi cu 9 - 16 membri, 3 octeti pentru seturi cu 17 - 24 membri, 4 octeti pentru seturi cu 25 - 32 membri, 8 octeti pentru seturi cu 33 - 64 membri. În cazul comparațiilor este insensibil la diferența între majuscule și minuscule.

## Capitolul 5. OPERATORI SI FUNCTII

MySQL nu impune ca o instructiune SELECT sa aiba clauza FROM, ceea ce faciliteaza exersarea utilizarii operatorilor si a functiilor, introducand expresii arbitrare in acest mod. Astfel, dupa invocarea programului *mysql*, se tasteaza expresia precedata de SELECT si urmata de punct si virgula.

Numele functiilor precum si a operatorilor de tip cuvânt pot fi specificate folosind atat majuscule cat si minuscule.

### 5.1. Operatori MySQL

Operatorii sunt folositi pentru a combina termenii din expresii, pentru a executa operatii aritmetice, pentru compararea valorilor, pentru a executa operatii la nivel de bit sau logice, pentru a identifica echivalente cu un model.

Operatorii au niveluri de precedenta variabile. Nivelurile sunt prezentate in lista urmatoare, in ordine descrescatoare:

| Nivel precedenta                                                                   | Operatori                                |
|------------------------------------------------------------------------------------|------------------------------------------|
|  | BINARY                                   |
|                                                                                    | NOT !                                    |
|                                                                                    | - (minus unar)                           |
|                                                                                    | * / %                                    |
|                                                                                    | + -                                      |
|                                                                                    | << >>                                    |
|                                                                                    | &                                        |
|                                                                                    |                                          |
|                                                                                    | < <= = <=> != <> IN IS LIKE REGEXP RLIKE |
|                                                                                    | BETWEEN                                  |
|                                                                                    | AND &&                                   |
|                                                                                    | OR                                       |

Operatorii situati pe aceeasi linie au aceeasi precedenta. Operatorii de la un anumit nivel de precedenta sunt evaluati de la stanga la dreapta. Operatorii de la un nivel de precedenta mai ridicat sunt evaluati inaintea operatorilor de la un nivel de precedenta mai redus.

Parantezele rotunde de deschidere si de inchidere se folosesc pentru a grupa parti ale unei expresii cu scopul de a anula precedenta prestabilita a operatorilor. De asemenea, parantezele mai pot fi folosite pur si simplu pentru a asigura claritatea vizuala, pentru a facilita citirea unei expresii.

MySQL foloseste urmatoarele tipuri de operatori: aritmetici, de comparatie, logici si la nivel de bit.

#### 5.1.1. Operatori aritmetici

Sunt folositi pentru a executa operatii aritmetice standard; lucreaza cu numere, nu cu siruri. Operatiile aritmetice cu valori NULL produc rezultat NULL.

Operatorii aritmetici folositi de MySQL sunt:

- a) + (adunare): calculeaza suma a doi operanzi; nu serveste ca operator de concatenare a sirurilor; acestea sunt convertite in numere inainte ca operatia aritmetica sa aiba loc. Sirurile care nu seamana cu numere sunt convertite in 0.
- b) – (minus) : calculeaza diferenta dintre operanzi atunci cand este folosit intre doi termeni ai unei expresii. Daca este folosit in fata unui singur termen, determina negativul operandului (ii schimba semnul).
- c) \* (inmultire): calculeaza produsul a doi operanzi.
- d) / (impartire): calculeaza catul a doi operanzi. Impartirea cu 0 (zero) produce un rezultat NULL.
- e) % (modulo): calculeaza restul impartirii unui operand  $m$  la alt operand  $n$ ; este similar cu MOD( $m,n$ ). Ca si in cazul impartirii, operatorul modulo cu un divizor egal cu zero returneaza NULL.

### 5.1.2. Operatori de comparatie

Operatorii de comparatie returneaza 1 (TRUE) in cazul in care comparatia este adevarata, respectiv 0 (FALSE) in caz contrar. Pot fi comparate atat numere cat si siruri. MySQL foloseste urmatoarele reguli atunci cand face comparatie intre doi operanzi (acestia fiind convertiti dupa necesitate):

- ✓ cu exceptia operatorului  $=$ , daca ambii operanzi sunt NULL, atunci rezultatul este NULL;
- ✓ daca ambii operanzi sunt siruri, ei vor fi comparati conform regulilor de comparare a sirurilor (din punct de vedere lexicografic); comparatiile intre siruri se executa folosind setul de caractere in vigoare de pe server;
- ✓ daca ambii operanzi sunt numere intregi, ei vor fi comparati din punct de vedere numeric, ca intregi;
- ✓ valorile hexazecimale sunt comparate sub forma de siruri binare atunci cand nu sunt comparate cu un numar;
- ✓ daca unul dintre operanzi este de tip TIMESTAMP sau DATETIME si celalalt o constanta, operanzii sunt comparati sub forma de valori TIMESTAMP;
- ✓ daca nu se aplica nici una dintre regulile precedente, operanzii sunt comparati ca valori numerice, in virgula mobila. In aceasta situatie este inclus si cazul in care se compara un sir cu un numar, sirul fiind convertit intr-un numar.

Operatorii de comparatie folositi de MySQL sunt:

- a) = (egalitate): are ca rezultat 1 daca operanzii sunt egali, respectiv 0 in caz contrar.
- b) != sau <> (inegalitate, diferit): determina rezultatul 1 daca operanzii sunt inegali, 0 in caz contrar.
- c) <= (mai mic sau egal): are ca rezultat 1 daca operandul din stanga este mai mic sau egal cu cel din dreapta, in caz contrar rezultatul este 0.
- d) < (mai mic): are ca rezultat 1 daca operandul din stanga este mai mic decat cel din dreapta, in caz contrar rezultatul este 0.
- e) >= (mai mare sau egal): are ca rezultat 1 daca operandul din stanga este mai mare sau egal cu cel din dreapta, in caz contrar rezultatul este 0.

- f) > (mai mare): are ca rezultat 1 daca operandul din stanga este mai mare decat cel din dreapta, in caz contrar rezultatul este 0.
- g) <=>: egalitate, accepta ca operanzi si valori NULL; este similar cu operatorul de egalitate, cu exceptia faptului ca atunci cand operanzii sunt egali si au valoarea NULL rezultatul returnat are valoarea 1.
- h) expresie IS NULL: are ca rezultat 1 daca valoarea operandului *expresie* este NULL, 0 in caz contrar.
- i) IS NOT NULL: determina rezultatul invers, fiind echivalent cu expresia: *NOT (expr IS NULL)*.
- j) expresie BETWEEN minim AND maxim: determina rezultatul 1 daca *minim* este mai mic sau egal cu *expresie* si daca *maxim* este mai mare sau egal cu *expresie*. Sunt utilizate urmatoarele reguli: daca *expresie* este intreg, atunci operanzii sunt comparati ca intregi; daca *expresie* este sir de caractere, operanzii sunt comparati ca siruri; compararea se face *case sensitive* sau nu, dupa cum *expresie* este sau nu un sir boolean; in celelalte cazuri, operanzii sunt comparati ca numere in virgula mobila.
- k) expresie IN (val1, val2, ...): determina rezultatul 1 daca *expresie* este una din valorile din lista, respectiv 0 in caz contrar.
- l) expresie NOT IN (val1, val2, ...): in acest caz situatia se inverseaza, fiind echivalent cu expresia: *NOT (expresie IN (val1, val2, ...))*

### 5.1.3. Operatori logici

Operatorii logici (booleeni) testeaza adevarul sau falsitatea expresiilor. Operatiile logice returneaza 1 pentru „adevarat”, respectiv 0 pentru „fals”. Operatorii logici interpreteaza operanzii diferiti de 0 ca „adevarat”, respectiv operanzii egali cu zero ca „fals”.

Operatorii logici accepta ca operanzi numere, asadar operanzii de tip sir de caractere sunt convertiti in numere inainte de evaluarea operatorului.

- a) ! sau NOT: negatia logica, are ca rezultat 1 daca operandul la care se aplica este 0 (*FALSE*), respectiv 0 daca operandul are valoarea 1 (*TRUE*). Exista insa o singura exceptie: NOT NULL este NULL.
- b) || sau OR: SAU logic, determina rezultatul 1 (*TRUE*) daca oricare dintre operanzi este adevarat (diferit de zero, sir vid sau NULL), respectiv 0 (*FALSE*) in caz contrar.
- c) && sau AND: SI logic, determina rezultatul 1 (*TRUE*) daca ambii operanzi sunt adevarati (diferiti de zero, sir vid sau NULL), respectiv 0 (*FALSE*) in caz contrar.

### 5.1.4. Operatori la nivel de bit

- a) | (OR): returneaza rezultatul operatiei OR (reuniune) la nivel de bit a operanzilor.
- b) & (SI): returneaza rezultatul operatiei AND (intersectie) la nivel de bit a operanzilor.
- c) <<: deplaseaza operandul stang spre stanga, cu un numar de pozitii de bit indicate de operandul drept. Deplasarea cu o cantitate negativa determina rezultatul 0. Utilizarea acestui operator este echivalenta cu inmultirea operandului din stanga cu 2 la o putere egala cu operandul din dreapta.
- d) >>: deplaseaza operandul stang spre dreapta, cu un numar de pozitii de bit indicate de operandul drept. Deplasarea cu o cantitate negativa determina rezultatul 0. Utilizarea acestui operator este

echivalenta cu impartirea operandului din stanga cu 2 la o putere egala cu operandul din dreapta si pastrarea partii intregi a rezultatului.

### 5.1.5. Operatori de conversie

**BINARY** : determina tratarea operatorului ca sir binar, astfel incat comparatiile care implica sirul devin sensibile la diferenta intre majuscule si minuscule. Daca operandul este un numar, el va fi convertit la sir, apoi comparatia este executata dupa criterii lexicografice, deoarece ambii operanzi sunt siruri.

## 5.2. Functii MySQL

MySQL utilizeaza un numar mare de functii predefinite. Cand se apeleaza o functie, lista argumentelor trebuie sa fie intre paranteze rotunde, iar intre numele functiei si paranteza de deschidere nu trebuie sa existe nici un spatiu. Daca se transmite functiei mai multe argumente, acestea vor fi despartite prin virgula.

### 5.2.1. Functii de manipulare a sirurilor de caractere

Functiile care au ca argumente siruri de caractere pot fi folosite in cadrul instructiunilor SELECT fara a specifica o tabela cu scopul de a obtine un rezultat, sau pot fi folosite asupra anumitor campuri din tabele in scopul concatenarii, ajustarii sau a modificarii acestora.

#### 5.2.1.1. Functii referitoare la lungime si functii de concatenare

Grupul de functii referitoare la lungime si de concatenare se concentreaza asupra lungimii sirurilor de caractere si asupra concatenarii acestora.

- ✓ **LENGTH(sir)**, **OCTET\_LENGTH(sir)**, **CHAR\_LENGTH(sir)**, **CHARACTER\_LENGTH(sir)** – returneaza lungimea sirului *sir* transmis ca argument.
- ✓ **CONCAT(sir1,sir2,...)** – returneaza un sir alcatuit prin concatenarea tuturor sirurilor primite ca argumente, sau NULL daca oricare dintre argumente este NULL.
- ✓ **CONCAT\_WS(separator,sir1,sir2,...)** – provine de la *concatenate with separator (concatenare cu separator)*. Separatorul poate fi orice caracter, cel mai adesea folosindu-se *spatiul alb*.

#### 5.2.1.2. Functii de ajustare si completare

MySQL pune la dispozitie functii pentru adaugarea si eliminarea caracterelor suplimentare (inclusiv a spatiilor albe) din sirurile de caractere.

- ✓ **RTRIM(sir)** – returneaza sirul *sir* fara spatiile de sfarsit, respectiv NULL daca *sir* este NULL.
- ✓ **LTRIM(sir)** - returneaza sirul *sir* fara spatiile de inceput, respectiv NULL daca *sir* este NULL.
- ✓ **TRIM([LEADING | TRAILING | BOTH] [trim\_sir] FROM sir)** – returneaza sirul *sir* fara instantele initiale/finale ale sirului *trim\_sir*. Daca este specificata optiunea LEADING, sunt eliminate aparitiile initiale ale sirului *trim\_sir*. Daca este specificata optiunea TRAILING, sunt eliminate aparitiile finale ale sirului *trim\_sir*. Daca este specificata optiunea BOTH, sunt

eliminate atat aparitiile initiale cat si cele finale ale sirului *trim\_sir*. Optiunea prestabilita este BOTH (daca nu este specificata nici una dintre optiuni). In cazul in care *trim\_sir* nu este precizat, atunci functia elimina spatiile.

- ✓ RPAD(sir, lungime, sir\_completare) – returneaza un sir alcatuit din sirul *sir*, completat la dreapta cu sirul *sir\_completare* pana cand se ajunge la un sir de *lungime* caractere.
- ✓ LPAD(sir, lungime, sir\_completare) - returneaza un sir alcatuit din sirul *sir*, completat la stanga cu sirul *sir\_completare* pana cand se ajunge la un sir de *lungime* caractere.

#### 5.2.1.3. Functii de gasire si pozitionare

LOCATE(subsir, sir) sau LOCATE(subsir, sir, pozitie) – returneaza pozitia primei aparitii a sirului *subsir* in cadrul sirului *sir*, respectiv 0 daca *subsir* nu apare in *sir*. Daca este precizat si argumentul *pozitie*, atunci se incepe cautarea sirului *subsir* de la pozitia respectiva. La cautare se face diferenta intre majuscule si minuscule.

#### 5.2.1.4. Functii care opereaza cu subsiruri

- ✓ SUBSTRING(sir, pozitie) sau SUBSTRING(sir, pozitie, lungime) sau SUBSTRING(sir FROM pozitie) sau SUBSTRING(sir FROM pozitie FOR lungime) – returneaza un subsir al sirului *sir*, incepand de la pozitia *poz*. Daca este dat un argument *lung*, returneaza un subsir care are un numar de caractere egal cu *lung*. In caz contrar, returneaza intreaga parte dreapta a sirului *sir*, incepand de la pozitia *poz*.
- ✓ SUBSTRING\_INDEX(sir, delim, n) – returneaza un subsir al sirului *sir*. Daca *n* este pozitiv, atunci functia gaseste cea de-a n-a aparitie a sirului de delimitare *delim*, apoi returneaza toate caracterele aflate la stanga acelui delimitator. Daca *n* este negativ, functia gaseste a n-a aparitie a sirului *delim*, numarand de la capatul din dreapta al sirului *sir*, apoi returneaza toate caracterele din dreapta delimitatorului respectiv. Daca *delim* nu se afla in *sir*, atunci va fi returnat intregul sir. Daca orice argument are valoarea NULL, functia va returna NULL.
- ✓ LEFT(sir, lungime) – returneaza primele n caractere (*n=lungime*) de la stanga la dreapta, ale sirului *sir*, respectiv intregul sir daca lungimea acestuia este mai mica decat numarul de caractere precizat in lungime. Daca *sir* este NULL, functia returneaza NULL, iar daca argumentul *lungime* este NULL sau mai mic decat 1, functia returneaza un sir vid.
- ✓ RIGHT(sir, lungime) - returneaza primele n caractere (*n=lungime*) de la dreapta la stanga, ale sirului *sir*, respectiv intregul sir daca lungimea acestuia este mai mica decat numarul de caractere precizat in lungime. Daca *sir* este NULL, functia returneaza NULL, iar daca argumentul *lungime* este NULL sau mai mic decat 1, functia returneaza un sir vid.

#### 5.2.1.5. Functii de modificare a sirurilor de caractere

- ✓ LCASE(sir) – returneaza sirul *sir* cu toate caracterele scrise cu minuscule. Daca *sir* este NULL, atunci returneaza NULL.
- ✓ LOWER(sir) – este sinonim cu LCASE
- ✓ UCASE(sir) – returneaza sirul *sir* cu toate caracterele scrise cu majuscule. Daca *sir* este NULL, atunci returneaza NULL.

- ✓ UPPER(sir) - este sinonim cu UCASE
- ✓ REPEAT(sir,n) – returneaza un sir format din sirul *sir*, repetat de *n* ori. Rezultatul este un sir vid daca *n* nu este pozitiv, respectiv NULL daca oricare dintre argumente este NULL.
- ✓ REPLACE(sir,din\_sir,in\_sir) – returneaza un sir alcatuit din sirul *sir*, unde toate aparitiile sirului *din\_sir* sunt inlocuite cu sirul *in\_sir*. Returneaza NULL daca oricare dintre argumente este NULL. Daca *in\_sir* este vid, atunci efectul consta in stergerea tuturor aparitiilor sirului *din\_sir*.

### 5.2.2. Functii numerice

Functiile numerice returneaza NULL daca se produce o eroare.

- ✓ ABS(x) – returneaza valoarea absoluta a lui x.
- ✓ ACOS(x) – returneaza valoarea  $\arccos(x)$ , respectiv NULL daca x nu se afla in domeniul cuprins intre -1 si 1.
- ✓ ASIN(x) – returneaza valoarea  $\arcsin(x)$ , respectiv NULL daca x nu se afla in domeniul cuprins intre -1 si 1.
- ✓ ATAN(x) – returneaza valoarea arctangentei lui x.
- ✓ CEILING(x) – returneaza cel mai mic intreg care nu este mai mic decat x.
- ✓ COS(x) – returneaza cosinusul lui x, unde x este masurat in radiani.
- ✓ COT(x) - returneaza cotangenta lui x, unde este masurat in radiani.
- ✓ EXP(x) – returneaza  $e^x$ , unde e reprezinta baza logaritmilor naturali.
- ✓ FLOOR(x) – returneaza cel mai mare intreg care nu este mai mare decat x.
- ✓ LOG(x) – returneaza logaritmul natural (in baza e) al lui x.
- ✓ LOG10(x) – returneaza logaritmul lui x in baza 10.
- ✓ MOD(m,n) – este identic cu  $m \% n$  (vezi operatori aritmetici).
- ✓ PI() – returneaza valoarea lui  $\pi$ .
- ✓ POW(x,y) – returneaza  $x^y$ , adica x ridicat la puterea y.
- ✓ POWER(x,y) – este sinonima cu functia POW.
- ✓ RAND(), RAND(n) – returneaza o valoare aleatoare cu virgula mobila, situata in intervalul 0.0 si 1.0. In cazul in care este transmis (este apelata cu) si parametrul *n*, atunci executa aceeasi operatie, insa toate apelurile cu acelasi valoare a parametrului *n* returneaza acelasi rezultat.
- ✓ ROUND(x), ROUND(x,d) – returneaza valoarea lui x, rotunjita la un intreg. Daca este precizat si al doilea parametru, valoarea returnata va fi rotunjita la un numar cu *d* cifre dupa virgula.
- ✓ SIGN(x) – returneaza -1, 0, sau 1, in functie de valoarea lui x (negativa, zero, sau pozitiva).
- ✓ SIN(x) - returneaza valoarea sinusului lui x, unde x este masurat in radiani.
- ✓ SQRT(x) – returneaza radacina patrata nenegativa a lui x.
- ✓ TAN(x) – returneaza valoarea tangentei lui x, unde x este masurat in radiani.
- ✓ TRUNCATE(x,d) – returneaza valoarea lui x, cu partea zecimala trunchiata la *d* cifre dupa virgula. Daca *d* este 0 atunci rezultatul nu are virgula sau parte zecimala. Daca *d* este mai mare decat numarul de cifre dupa virgula al argumentului x, partea zecimala este completata cu zerouri finale, pana la dimensiunea dorita.

### 5.2.3. Functii de tip data si ora

- ✓ ADDDATE(data, INTERVAL expr interval) – este sinonima cu DATE\_INTERVAL().

- ✓ CURDATE() – returneaza data curenta sub forma de sir in formatul "AAAA-LL-ZZ", respectiv sub forma de numar in formatul AAAALLZZ, conform contextului in care este folosita.
- ✓ CURRENT\_DATE – este sinonima cu CURDATE().
- ✓ CURTIME() – returneaza ora curenta sub forma unui sir "hh:mm:ss", respectiv sub forma de numar in formatul hhmmss, conform contextului in care este folosita.
- ✓ CURRENT\_TIME – este sinonima cu CURTIME().
- ✓ CURRENT\_TIMESTAMP – este sinonima cu functia NOW().
- ✓ DATE\_ADD(data, INTERVAL expr interval) – pentru o valoare de tip data sau data-ora, adauga la aceasta un interval de timp si returneaza rezultatul; *expr* specifica valoarea orei care trebuie adaugata la data, iar interval specifica modul de interpretare al intervalului.

Rezultatul este o valoare DATE daca *data* este o valoare de tip DATE, altfel rezultatul va fi o valoare DATETIME. Rezultatul este NULL daca *data* nu este o data corecta.

Tabelul urmator prezinta valorile permise pentru argumentul *interval*, semnificatiile lor si formatul in care trebuie specificate valorile pentru fiecare tip de interval.

Cuvantul cheie INTERVAL si specificatorii interval pot fi mentionati folosind atat majuscule, cat si minuscule. Expresia *expr* care se adauga la data poate fi specificata sub forma de numar sau ca sir. Caracterele de delimitare sunt arbitrare si pot fi alcatuite din orice semn de punctuatie.

| Tip interval  | Semnificatie                 | Formatul valorii |
|---------------|------------------------------|------------------|
| SECOND        | Secunde                      | ss               |
| MINUTE        | Minute                       | mm               |
| HOURL         | Ore                          | hh               |
| DAY           | Zile                         | ZZ               |
| MONTH         | Luni                         | LL               |
| YEAR          | Ani                          | AA               |
| MINUTE_SECOND | Minute si secunde            | "mm:ss"          |
| HOURL_MINUTE  | Ore si minute                | "hh:mm"          |
| HOURL_SECOND  | Ore, minute si secunde       | "hh:mm:ss"       |
| DAY_HOURL     | Zile si ore                  | "ZZ hh"          |
| DAY_MINUTE    | Zile, ore si minute          | "ZZ hh:mm"       |
| DAY_SECOND    | Zile, ore, minute si secunde | "ZZ hh:mm:ss"    |
| YEAR_MONTH    | Ani si luni                  | "AA-LL"          |

- ✓ DATE\_FORMAT(data, format) – formateaza o valoare de tip data sau data-ora in conformitate cu sirul de formatare si returneaza sirul rezultat. Functia se poate folosi pentru a reformata valorile DATE sau DATETIME din forma folosita de MySQL cu scopul de a obtine orice format se doreste.

Tabelul de mai jos prezinta specificatorii disponibili care pot fi folositi in formatare:

| Specificator | Semnificatie                                                |
|--------------|-------------------------------------------------------------|
| %S, %s       | Secunda in format numeric cu doua cifre (interval 00 .. 59) |

|        |                                                                    |
|--------|--------------------------------------------------------------------|
| %i     | Minut in format numeric cu doua cifre (interval 00 .. 59)          |
| %H     | Ora in format numeric cu doua cifre, 24 de ore (interval 00 .. 23) |
| %h, %l | Ora in format numeric cu doua cifre, 12 ore (interval 01 .. 12)    |
| %k     | Ora in format numeric, 24 de ore (interval 0 .. 23)                |
| %l     | Ora in format numeric, 12 ore (interval 1 .. 12)                   |
| %T     | Ora din zi, in format 24 de ore (hh:mm:ss)                         |
| %r     | Ora din zi, in format 12 de ore (hh:mm:ss AM sau hh:mm:ss PM)      |
| %p     | AM sau PM                                                          |
| %W     | Numele zilei din saptamana (Sunday, Monday, ... Saturday)          |
| %a     | Numele abreviat al zilei din saptamana (Sun, Mon, ... Sat)         |
| %d     | Ziua din luna, in format numeric cu doua cifre (01 .. 31)          |
| %e     | Ziua din luna, in format numeric (1 .. 31)                         |
| %D     | Ziua din luna, cu sufixele limbii engleze (1st, 2nd, ...)          |
| %w     | Ziua din saptamana, in format numeric (0=duminica, 1=luni, ...)    |
| %j     | Ziua din an, in format numeric cu trei cifre (001 .. 366)          |
| %U     | Saptamana (0 .. 52), unde duminica este prima zi a saptaminii      |
| %u     | Saptamana (0 .. 52), unde luni este prima zi a saptaminii          |
| %M     | Numele lunii (January, February, ... December)                     |
| %b     | Numele lunii, in forma abreviata (Jan, Feb, ... Dec)               |
| %m     | Luna in format numeric cu doua cifre (01 .. 12)                    |
| %c     | Luna in format numeric (1 .. 12)                                   |
| %Y     | Anul in format numeric cu patru cifre                              |
| %y     | Anul in format numeric cu doua cifre                               |

- ✓ DATE\_SUB(data, INTERVAL expr interval) – executa operatii aritmetice cu date in aceeași maniera ca și DATE\_ADD(), cu excepția faptului că *expr* se scade din valoarea datei.
- ✓ DAYNAME(data) – returnează un șir care conține numele zilei din săptămână pentru o valoare de tip data.
- ✓ DAYOFMONTH(data) – returnează valoarea numerică a zilei din luna corespunzătoare valorii de tip data, unde valoarea este cuprinsă între 1 și 31.
- ✓ DAYOFWEEK(data) - returnează valoarea numerică a zilei din săptămână corespunzătoare valorii de tip data, având domeniul între 1 (duminică) și 7 (sambătă).
- ✓ DAYOFYEAR(data) - returnează valoarea numerică a zilei din an corespunzătoare valorii de tip data, valoarea este cuprinsă între 1 și 366.
- ✓ EXTRACT(interval FROM dataora) – returnează partea din valoarea data-ora indicată de *interval*, care poate fi oricare dintre specificatorii de interval permisi pentru DATE\_ADD(). Poate fi folosit și cu valori DATE, în acest caz rezultatul pentru extragerea orei fiind NULL.
- ✓ FROM\_DAYS(n) – dacă este data o valoare numerică reprezentând numărul de zile scurse de la anul 0, returnează data corespunzătoare.
- ✓ HOUR(ora) – returnează valoarea numerică a orei pentru valoarea de tip ora (interval 0 – 23).
- ✓ MINUTE(ora) - returnează valoarea numerică a minutelor pentru valoarea de tip ora (interval 0 – 59).
- ✓ MONTH(data) - returnează valoarea numerică a lunii din an aferente valorii de tip data, fiind cuprinsă în intervalul 1 – 12.

- ✓ MONTHNAME(data) - returneaza un sir care contine numele lunii din an aferente valorii de tip data.
- ✓ NOW() – returneaza data si ora curenta sub forma de sir "AAAA-LL-ZZ hh:mm:ss", respectiv sub forma de numar in formatul AAAALLZZhhmmss, conform contextului in care este folosita.
- ✓ PERIOD\_ADD(perioada, n) – adauga *n* luni la valoarea de tip perioada si returneaza rezultatul. Formatul rezultatului este AAAALL, iar formatul argumentului *perioada* poate fi AAAALL sau AALL. Niciuna nu este valoare de tip data.
- ✓ PERIOD\_DIFF(perioada1, perioada2) – preia diferenta intre argumentele de tip perioada si returneaza numarul de luni cuprins intre acestea. Argumentele pot fi de tipul AAAALL sau AALL. Niciunul nu este valoare de tip data.
- ✓ QUARTER(data) – returneaza valoarea numerica a trimestrului din an aferent valorii de tip data, avand domeniul cuprins intre 1 si 4.
- ✓ SECOND(ora) - returneaza valoarea numerica a numarului de secunde pentru valoarea de tip ora, fiind cuprinsa in intervalul 0 si 59.
- ✓ SEC\_TO\_TIME(secunde) – daca este dat un numar de secunde, returneaza valoarea corespondenta a orei sub forma de sir "hh:mm:ss", respectiv sub forma de numar in formatul hhmmss, conform contextului in care este folosita functia.
- ✓ SUBDATE(data, INTERVAL expr interval) – este sinonima cu functia DATE\_SUB().
- ✓ SYSDATE() – este sinonima cu functia NOW().
- ✓ TIME\_FORMAT(ora,format) – formateaza valoarea de tip ora conform cu sirul de formatare si returneaza sirul rezultat. Sirul de formatare este similar cu cel folosit de catre functia DATE\_FORMAT(), diferenta fiind faptul ca singurii specificatori care pot fi folositi sunt cei referitori la ora. Utilizarea altor specificatori determina rezultat NULL sau 0.
- ✓ TIME\_TO\_SEC(ora) – daca este data o valoare *ora* care reprezinta timpul scurs, functia returneaza un numar care reprezinta numarul corespunzator de secunde.
- ✓ TO\_DAYS(data) – returneaza o valoare numerica reprezentand valoarea de tip data convertita la numarul de zile scurs de la anul 0.
- ✓ WEEK(data), WEEK(data,prima) – cand este apelata cu un singur argument, returneaza un numar care reprezinta saptamana din an aferenta valorii de tip data, in domeniul cuprins intre 0 si 52. Cand este apelata cu doua argumente, returneaza acelasi rezultat, insa argumentul *prima* indica prima zi din saptamana; daca acesta ia valoarea 0 atunci saptamana incepe cu *duminica*, iar daca este 1 saptamana incepe cu *luni*.
- ✓ WEEKDAY(data) – returneaza valoarea numerica a zilei din saptamana aferenta valorii de tip data. Valorile din saptamana sunt cuprinse intre 0 (pentru luni) si 6 (pentru duminica).
- ✓ YEAR(data) – returneaza valoarea numerica a anului pentru valoarea de tip data, in domeniul cuprins intre 1000 si 9999.

#### 5.2.4. Functii de sumar

Functiile de sumar sunt cunoscute sub numele de *functii agregat*. Acestea calculeaza o valoare unica, pe baza unui grup de valori.

- ✓ AVG(expr) – returneaza valoarea medie a argumentului *expr* pentru toate valorile diferite de NULL din randurile selectate.

- ✓ COUNT(expr) – cu orice argument diferit de \* returneaza un numar al valorilor diferite de NULL din setul de rezultate. Cu argumentul \* returneaza numarul tuturor randurilor din setul de rezultate, indiferent daca randurile contin sau nu valori NULL.
- ✓ MAX(expr) - returneaza valoarea maxima a argumentului expr pentru toate valorile diferite de NULL din randurile selectate. Functia se poate folosi si cu siruri, caz in care returneaza valoarea cea mai mare din punct de vedere lexicografic
- ✓ MIN(expr) – returneaza valoarea minima a argumentului expr pentru toate valorile diferite de NULL din randurile selectate. Functia se poate folosi si cu siruri, caz in care returneaza valoarea cea mai mica din punct de vedere lexicografic.
- ✓ STD(expr) – returneaza abaterea standard a argumentului *expr* pentru toate valorile diferite de NULL din randurile selectate.
- ✓ STDEV(expr) – este sinonima cu STD().
- ✓ SUM(expr) – returneaza suma argumentelor *expr* pentru toate valorile diferite de NULL din randurile selectate.

### 5.2.5. Functii diverse

- ✓ BENCHMARK(contor,expr) – evalueaza in mod repetat expresia *expr* de *contor* ori. Valoarea returnata este intotdeauna 0, insa valoarea de interes o reprezinta timpul scurs intre momentul afisarii rezultatului interogarii si momentul afisarii valorii returnate. Timpul scurs reprezinta numai un indicator aproximativ al vitezei de evaluare a expresiei de catre server, el fiind influentat de factori precum incarcarea serverului. Pentru a obtine un rezultat reprezentativ trebuie rulata interogarea de mai multe ori.
- ✓ DATABASE() – returneaza un sir care contine numele bazei de date curente, respectiv un sir vid daca nu exista nici o baza de date curenta.
- ✓ DECODE(sir, parola) – daca este dat un sir criptat *sir*, obtinut ca rezultat al unui apel al functiei DECODE(), functia il decripteaza folosind sirul *parola* si returneaza rezultatul.
- ✓ ENCODE(sir,parola) – cripteaza sirul *sir* folosind sirul *parola* si returneaza rezultatul sub forma de sir binar. Sirul poate fi decodificat folosind functia DECODE().
- ✓ ENCRYPT(sir), ENCRYPT(sir,salt) – cripteaza sirul *sir* si returneaza sirul rezultat. Criptarea este ireversibila. In cazul in care se specifica si argumentul *salt* (care trebuie sa fie un sir format din doua caractere pentru versiunile mai vechi de MySQL), rezultatul criptarii sirului *sir* va fi de fiecare data acelasi. ENCRYPT() va returna intotdeauna NULL.
- ✓ LAST\_INSERT\_ID(), LAST\_INSERT\_ID(expr) – fara nici un argument, functia returneaza valoarea AUTO\_INCREMENT cel mai recent generata in timpul sesiunii curente a serverului, respectiv 0 daca nu a fost generata o asemenea valoare. Cu un argument, functia este destinata utilizarii intr-o instructiune UPDATE. Rezultatul este tratat in acelasi mod ca si o valoare generata automat, care este utila pentru generarea secventelor.
- ✓ LOAD\_FILE(ume\_fisier) – citeste fisierul *ume\_fisier* si returneaza continutul sub forma de sir. Fisierul trebuie sa fie localizat pe server, trebuie specificata calea de acces absoluta si trebuie sa fie accesibil la citire.
- ✓ MD5(sir) – calculeaza o suma de control a sirului *sir*, bazata pe algoritmul MD5 Message-Digest al firmei RSA Data Security, Inc. Valoarea returnata este un sir alcatuit din 32 de cifre hexazecimale.

- ✓ PASSWORD(sir) – daca este dat un sir sir, calculeaza si returneaza un sir parola criptat, operatia de criptare fiind ireversibila.
- ✓ SESSION\_USER() – este sinonima cu functia USER().
- ✓ SYSTEM\_USER – este sinonima cu functia USER.
- ✓ USER – returneaza un sir cu numele utilizatorului curent, sub forma *utilizator@gazda*.
- ✓ VERSION – returneaza un sir care reprezinta versiunea programului server MySQL.

Institutul Multimedia Româno-Elvetian

## Capitolul 6. CREAREA, MODIFICAREA SI STERGEREA BAZELOR DE DATE SI A TABELELOR

Un utilizator poate crea o baza de date doar daca are privilegiul CREATE. Astfel, administratorul poate crea un cont de utilizator cu privilegii de creare, in caz contrar fiind nevoit sa creeze el insusi baza de date de care are nevoie utilizatorul.

### 6.1. Crearea si selectarea bazelor de date

Presupunem ca administratorul creeaza un cont de utilizator cu privilegii extinse (de creare a bazelor de date si a tabelelor) cu numele *noname* si parola *paswd*. Acesta se va putea conecta la serverul *localhost*. Baza de date asupra careia are toate drepturile se numeste *mybd*. Administratorul MySQL va introduce urmatoarea comanda:

```
GRANT ALL ON mybd.* TO noname@localhost IDENTIFIED BY "paswd"
```

#### 6.1.1. Crearea bazelor de date

Dupa conectarea la serverul MySQL, utilizatorul va putea crea o baza de date utilizand comanda:

```
CREATE DATABASE nume_baza_de_date
```

Instructiunea esueaza daca baza de date exista deja sau daca nu exista privilegiul de a o crea.

#### 6.1.2. Afisarea bazelor de date

Pentru a verifica existenta unei baze de date se foloseste comanda:

```
SHOW DATABASES [LIKE sablon]
```

Instructiunea enumera bazele de date disponibile (la care utilizatorul respectiv are acces) pe gazda serverului.

#### 6.1.3. Selectarea bazelor de date

Inainte de utilizarea oricarei baze de date (efectuarea de operatii asupra ei) trebuie folosita in mod obligatoriu comanda USE, care selecteaza baza de date si o transforma in baza de date curenta.

```
USE nume_baza_de_date
```

Instructiunea USE esueaza daca baza de date nu exista sau daca nu sunt privilegii pentru a obtine accesul la ea.

## 6.2. Crearea tabelelor

În procesul de creare a unui tabel trebuie să se stabilească numele tabelului, numele câmpurilor și caracteristicilor (definițiile) acestora. Numele trebuie să fie simbolice, concise și relevante.

### 6.2.1. Crearea tabelelor

Comanda de creare a unei tabeli este CREATE TABLE, cu sintaxa următoare:

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] nume_tabel  
[(definire)]  
[optiuni_tabel]  
[[IGNORE | REPLACE] instructiune_select]
```

Unde:

- ✓ *nume\_tabel* este numele tabelului care va fi creat;
- ✓ *definire* conține definițiile coloanelor tabelului (urmează a fi detaliat);
- ✓ *optiuni\_tabel* permite specificarea unor opțiuni pentru tabelul creat;
- ✓ *instructiune\_select* permite crearea tabelului pe baza rezultatului întors de o interogare SELECT

Definirea coloanelor are următoarea sintaxă:

```
{ declaratie_coloana  
  | PRIMARY KEY (coloane_index)  
  | KEY [nume_index] [coloane_index]  
  | INDEX [nume_index] [coloane_index]  
  | UNIQUE INDEX [nume_index] [coloane_index]  
  | [CONSTRAINT simbol] FOREIGN KEY nume_index (coloane_index)  
  | [definitie_referinta]  
  | CHECK (expresie)  
}
```

în care *declaratie\_coloana* are sintaxa:

```
nume_coloana tip_coloana  
[NOT NULL | NULL] [DEFAULT valoare_implicita]  
[AUTO_INCREMENT] [PRIMARY KEY] [definitie_referinta]
```

și *definitie\_expresie* are sintaxa:

```
REFERENCES nume_tabel [(coloane_index)]  
[MATCH FULL | MATCH PARTIAL]  
[ON DELETE optiune_referinta]  
[ON UPDATE optiune_referinta]
```

iar *optiune\_referinta*:

```
{RESTRICT | CASCADE | SET NULL | NO ACTION | SET DEFAULT}
```

Asadar, instructiunea CREATE TABLE creeaza un tabel nou cu numele *nume\_tabel*. In cazul in care este specificata optiunea TEMPORARY, atunci tabelul exista numai pana la terminare in conditii normale sau anormale a conexiunii client curente, sau pana la executarea unei instructiuni DROP TABLE. Tabelele temporare sunt vizibile doar pentru clientii care le-au creat.

In definirea tabelului poate fi o declaratie de coloana sau o definitie de index. O declaratie de coloana incepe cu numele coloanei si tipul, dupa care urmeaza o serie de cuvinte cheie optionale. Tipul de coloana poate fi oricare dintre cele prezentate. Cuvintele cheie care pot aparea sunt urmatoarele:

- ✓ NULL sau NOT NULL: coloana poate contine sau nu valori NULL. Daca niciuna dintre aceste optiuni nu este specificata, optiunea prestabilita este NULL;
- ✓ DEFAULT valoare\_prestabilita: determina valoarea prestabilita a coloanei. Nu se poate folosi pentru tipurile TEXT si BLOB. Pentru coloanele care accepta valori NULL, valoarea prestabilita este NULL. Pentru cele care nu accepta valoarea NULL, daca coloanele sunt numerice atunci valoarea prestabilita este 0 (cu exceptia AUTO\_INCREMENT, care atribuie urmatorul numar din secventa de numere aferente coloanei respective); daca coloanele sunt de tip data si ora (exclus TIMESTAMP), valoarea prestabilita este "0000-00-00" pentru cele de tipul DATE sau "00:00:00" pentru cele de tipul TIME; pentru TIMESTAMP valoarea prestabilita este data si ora curenta; pentru coloanele de tip sir (exclus ENUM) valoarea prestabilita este situl vid; pentru ENUM, valoarea prestabilita este primul element din enumerare.
- ✓ AUTO\_INCREMENT: se aplica numai coloanelor cu valori numerice intregi. Valorile coloanelor care au acest atribut pleca de la valoarea initiala 1 sunt incrementate automat cu o unitate la fiecare instructiune INSERT sau UPDATE. Valoarea inserata efectiv este cu o unitate mai mare decat valoarea curenta maxima din coloana. Coloanele asupra carora se aplica acest cuvint cheie trebuie specificate sub forma de index UNIQUE sau PRIMARY KEY si trebuie sa fie de tipul NOT NULL. Intr-un tabel poate exista cel mult o coloana AUTO\_INCREMENT.
- ✓ PRIMARY KEY: arata ca respectiva coloana este de tipul PRIMARY KEY.
- ✓ UNIQUE: arata ca respectiva coloana este un index UNIQUE.

La crearea unui tabel, se poate specifica o lista de optiuni care poate contine una sau mai multe dintre optiunile urmatoare:

- ✓ AUTO\_INCREMENT = n : prima valoare AUTO\_INCREMENT care va fi generata pentru tabel.
- ✓ CHECKSUM = {0 | 1} : daca optiunea are valoarea 1 se foloseste o suma de control pentru fiecare rand de tabel.
- ✓ COMMENT = "sir"
- ✓ MAX\_ROWS
- ✓ MIN\_ROWS
- ✓ ROW\_FORMAT
- ✓ [DEFAULT] CHARACTER SET nume\_set\_de\_caractere

Daca este specificata o clauza *instructiune\_select* sub forma unei interogari SELECT, atunci tabelul este creat folosind continutul setului de rezultate returnat de interogare.

### 6.2.2. Afisarea tabelelor dintr-o baza de date

Daca se doreste afisarea numelor tabelelor existente intr-o baza de date, se foloseste comanda:

```
SHOW TABLES [FROM nume_baza_de_date [LIKE sablon]]
```

### 6.3. Indexarea tabelelor

Indexarea tabelelor constituie o metoda des utilizata pentru cautari eficiente in tabele, in sensul minimizarii timpului de acces, mai ales la utilizarea clauzei WHERE in cadrul instructiunii SELECT. Indexarea este esentiala in cazul in care tabelele contin un numar mare sau foarte mare de inregistrari si in cazul cautarilor care implica utilizarea mai multor tabele.

MySQL permite indexarea coloanelor individual sau in combinatii de mai multe coloane. De asemenea, se poate utiliza pentru indexare doar o parte a unei coloane (mai precis primele caractere ale acesteia), cu exceptia coloanelor de tip SET si ENUM. Nu pot exista doi indecsi cu acelasi nume.

Indecsii pot fi creati odata cu tabelul, utilizand instructiunea CREATE TABLE, sau pot fi adaugati ulterior utilizand instructiunile CREATE INDEX si ALTER TABLE.

MySQL permite crearea a trei tipuri de indecsi: PRIMARY KEY, UNIQUE si INDEX. Indecsii de tip PRIMARY KEY si UNIQUE nu admit valori duplicate ale inregistrarilor.

#### 6.3.1. Indexarea direct in momentul crearii

Un index poate fi creat prin specificarea anumitor clauze in instructiunea de creare a unui tabel CREATE TABLE.

Clauzele PRIMARY KEY, UNIQUE, INDEX si KEY se refera la indexuri. PRIMARY KEY si UNIQUE specifica indexuri ce contin doar valori unice, in timp ce INDEX si KEY (care sunt sinonime) specifica indexuri care pot contine si valori duplicate.

#### 6.3.2. Indexarea tabelelor deja create

Se utilizeaza instructiunea CREATE INDEX, cu sintaxa:

```
CREATE [UNIQUE] INDEX nume_index ON nume_tabel (coloane_index)
```

Instructiunea adauga un index cu numele *nume\_index* la tabelul *nume\_tabel*; este tratata ca si instructiunile ALTER TABLE ADD INDEX sau ALTER TABLE ADD UNIQUE.

#### 6.3.3. Afisarea informatiilor despre indecsii unui tabel

Instructiunea SHOW INDEX afiseaza informatii despre indexurile unui tabel.

```
SHOW INDEX FROM [nume_bd.]nume_tabel
```

Instructiunea SHOW KEYS este sinonima cu SHOW INDEX.

Datele de iesire ale functiei SHOW INDEX contin urmatoarele coloane:

- ✓ Table: numele tabelului;
- ✓ Non\_unique: are valoarea 1 daca indexul poate contine valori duplicate, respectiv 0 daca indexul nu poate contine asemenea valori;
- ✓ Key\_name: numele indexului;
- ✓ Seq\_in\_index: numarul coloanei din index (coloanele sunt numerotate incepand cu 1);
- ✓ Column\_name: numele coloanei;
- ✓ Collation: ordinea de sortare a coloanelor in cadrul indexului. Valorile pot fi A (ascending - crescator), D (descending - descrescator) sau NULL (fara sortare);
- ✓ Cardinality: numarul de valori unice din index;
- ✓ Sub\_part: lungimea prefixului, daca este indexat numai un prefix al coloanei. Are valoarea NULL daca este indexata intreaga coloana;
- ✓ Packed: modul in care este impachetata cheia. Daca nu este impachetata are valoarea NULL;
- ✓ Comment: comentarii referitoare la index.

### 6.3.4. Stergerea unui index

Stergerea unui index se realizeaza utilizand instructiunea:

```
DROP INDEX nume_index ON nume_tabel
```

Aceasta sterge indexul *nume\_index* din tabelul *nume\_tabel*. Alternativ, se poate folosi si instructiunea: ALTER TABLE nume\_tabel DROP INDEX nume\_index.

## 6.4. Modificarea structurii tabelelor

Instructiunea ALTER TABLE permite redenumirea sau modificarea structurii tabelelor; se foloseste atunci cand se constata ca s-a omis introducerea unui camp intr-un tabel sau ca numele, tipul sau attributele unui camp trebuie modificate.

```
ALTER [IGNORE] TABLE nume_tabel  
specificator [,specificator ...]
```

unde *specificator* poate fi:

- ✓ ADD [COLUMN] declaratie\_coloane [FIRST | AFTER nume\_coloana]
  - permite adaugarea de coloane noi tabelului. Coloana nou introdusa va fi prima daca se foloseste optiunea FIRST, sau va fi plasata dupa o coloana specificata daca se foloseste optiunea AFTER. Daca nu este specificata nici o optiune, coloana adaugata va fi ultima.
- ✓ ADD INDEX [nume\_index] (coloane\_index)
  - permite adaugarea unui index la tabel., bazat pe coloanele specificate in *coloane\_index*. In cazul in care acestea (coloanele index) sunt mai multe, atunci ele trebuie separate prin virgula. Daca numele indexului nu este specificat, atunci se va atribui automat un nume in functie de numele primei coloane indexate.
- ✓ ADD UNIQUE [nume\_index] (coloane\_index)
  - permite adaugarea unui index cu valori unice la tabel. Optiunile *nume\_index* si *coloane\_index* sunt specificate ca si in cazul instructiunii ADD INDEX.
- ✓ ADD PRIMARY KEY (coloane\_index)

- permite adaugarea unei chei primare (un index de tip PRIMARY KEY) la tabel. Optiunea *coloane\_index* este specificata ca si in cazul instructiunii ADD INDEX. Daca exista deja o cheie primara, va rezulta o eroare.
- ✓ ALTER [COLUMN] nume\_coloana {SET DEFAULT valoare | DROP DEFAULT}
  - permite modificarea valorii implicite a unei coloane, fie prin inlocuirea ei cu alta (prin folosirea optiunii SET DEFAULT *valoare*), fie prin stergerea ei (prin folosirea optiunii DROP DEFAULT).
- ✓ CHANGE [COLUMN] nume\_coloana declaratie\_noua\_coloana
  - permite modificarea numelui si definitia unei coloane. *Nume\_coloana* reprezinta numele coloanei care va fi modificata, iar *declaratie\_noua\_coloana* este declaratia noua a coloanei, care cuprinde inclusiv si numele nou pentru coloana (eventual poate fi acelasi cu cel vechi) si declaratia cu tipul si attributele acesteia.
- ✓ MODIFY [COLUMN] declaratie\_noua
  - permite modificarea declaratiei unei coloane. Declaratie coloanei se face folosind acelasi format ca si cel pentru decierea de coloana din cadrul instructiunii CREATE TABLE.
- ✓ DROP [COLUMN] nume\_coloana
  - permite stergerea unei coloane date din tabel. Daca coloana face parte din vreun index sau mai multe, atunci va fi eliminata si acestea. In cazul in care vor fi sterse toate coloanele dintr-un index, atunci indexul va fi de asemenea sters.
- ✓ DROP PRIMARY KEY
  - permite stergerea cheii primare din tabel. Daca in tabel nu exista nici o astfel de cheie primara, va fi stearsa prima cheie UNIQUE din tabel.
- ✓ DROP INDEX nume\_index
  - permite stergerea indexului cu numele dat din tabel.
- ✓ RENAME [TO] nume\_tabel\_nou
  - permite redenumirea tabelului. Cuvantul cheie TO este optional.
- ✓ optiuni\_tabel
  - permite specificarea unor optiuni pentru tabel asemenator cu cele precizate in cadrul instructiunii CREATE TABLE.

Cuvantul cheie IGNORE actioneaza in momentul in care in tabelul nou exista valori duplicate ale cheilor unice.

## 6.5. Stergerea tabelelor si a bazelor de date

MySQL are instructiuni care permit stergerea tabelelor si a bazelor de date, cu mentiunea ca acestea pot fi sterse doar de catre utilizatori autorizati in acest sens.

### 6.5.1. Stergerea tabelelor

Stergerea unui tabel se face cu instructiunea:

```
DROP TABLE [IF EXISTS] nume_tabel
```

Daca este folosita clauza IF EXISTS, stergerea unui tabel inexistent din baza de date nu genereaza o eroare.

### 6.5.2. Stergerea bazelor de date

Stergerea unei baze de date se face prin utilizarea comenzii urmatoare:

```
DROP DATABASE [IF EXISTS] nume_baza_de_date
```

Eliminarea unei baze de date este o operatie ireversibila. O baza de date este reprezentata intr-un catalog amplasat in catalogul de date. Daca in catalogul respectiv au fost inserate si alte fisiere ce nu reprezinta tabele, fisierele respective nu vor fi sterse de instructiunea DROP DATABASE, caz in care nici catalogul bazei de date nu va fi sters. In schimb, vor fi sterse toate fisierele corespunzatoare tabelor continute in baza de date respectiva.

## Capitolul 7. REGASIREA INFORMATIILOR IN TABELE

Informatiile stocate in tabelele componente ale unei baze de date trebuie sa poata fi regasite sub diverse forme.

### 7.1. Instructiunea SELECT

Extragerea randurilor din tabelele specificate se face cu ajutorul instructiunii SELECT, care are sintaxa:

```
SELECT [optiuni_selectare] obiecte_selectate  
[INTO OUTFILE nume_fisier optiuni_export]  
[FROM lista_tabele]  
[WHERE conditii]  
[GROUP BY lista_coloane]  
[HAVING expresie]  
[ORDER BY nume_col [ASC|DESC]]  
[LIMIT [initial],numar_inregistrari]  
PROCEDURE [nume_procedura]
```

Toate clauzele sunt optionale, in afara de cuvantul cheie SELECT si de clauza *obiecte\_selectate*, care accepta instructiuni ce pur si simplu evalueaza expresii.

Clauza *optiuni\_selectate* poate contine urmatoarele optiuni:

- ✓ ALL (valoare implicita) care determina returnarea tuturor randurilor;
- ✓ DISTINCT si DISTINCTROW specifica eliminarea randurilor duplicate din setul de rezultate intors de instructiunea SELECT.
- ✓ HIGH\_PRIORITY specifica faptul ca instructiunea dobandeste o prioritate mai ridicata, daca in mod normal ar fi trebuit sa astepte dupa alte instructiuni precum INSERT si UPDATE. Se foloseste doar pentru instructiuni SELECT ce trebuie executate rapid, deoarece reduce viteza de executie a instructiunilor de scriere.

Clauza *obiecte\_selectate* precizeaza coloanele de iesire ce trebuie returnate. In cazul in care se doreste afisarea tuturor coloanelor unui tabel, se foloseste notatia speciala (caracterul) "\*". Daca se doreste ca doar o parte din coloane sa fie afisate, atunci numele acestora trebuie enumerate sub forma de lista, fiind separate prin virgule.

Coloanele de iesire pot fi referinte la coloanele din tabel sau pot fi expresii. Orice coloana poate primi un alias, in acest caz folosindu-se sintaxa: *AS nume\_alias*. Astfel, aliasul devine numele coloanei de iesire si se poate face referire la acesta in clauzele GROUP BY, ORDER BY si HAVING.

Rezultatul unei interogari SELECT poate fi scris intr-un fisier folosind clauza INTO OUTFILE urmata de numele fisierului in care urmeaza sa se scrie. Conditia esentiala este ca utilizatorul sa aiba privilegiul FILE si fisierul in care se scrie sa nu existe.

Clauza FROM indica numele tabelului sau a tabelelor din care trebuie selectate randurile. Clauza poate lipsi atunci cand nu se doreste selectarea datelor din tabele, altfel este obligatorie.

Tabelele pot primi aliasuri folosindu-se sintaxa *nume\_tabel* *nume\_alias* sau *nume\_tabel AS* *nume\_alias*.

Clauza PROCEDURE denumeste o procedura careia ii vor fi trimise datele din setul de rezultate inainte de a i se returna clientului un set de rezultate.

## 7.2. Clauza WHERE

Exista cazuri in care trebuie extrase din tabel doar anumite inregistrari. Prin intermediul clauzei WHERE se specifica o expresie care aplicata randurilor selectate din tabel, elimina acele randuri care nu satisfac criteriile date de expresie.

In cadrul expresiilor ce urmeaza clauzei WHERE pot fi folositi operatori de comparare si cei logici, cu urmatoarele semnificatii:

| Operator | Semnificatie         |
|----------|----------------------|
| =        | Egal cu              |
| !=       | Diferit de           |
| <=       | Mai mic sau egal cu  |
| <        | Mai mic decat        |
| >=       | Mai mare sau egal cu |
| >        | Mai mare decat       |

Mai exista operatorul BETWEEN, folosit in special in cazul comparatiilor intre numere intregi sau date calendaristice, deoarece cauta rezultate cuprinse intre o valoare minima si una maxima.

Alti operatori logici care permit efectuarea comparatiilor multiple in cadrul aceleiasi clauze WHERE sunt AND (toate comparatiile din cadrul clauzei trebuie sa fie adevarate pentru ca rezultatul sa fie extras) si OR (daca una dintre comparatii este adevarata, rezultatul va fi extras).

Totodata, se poate folosi operatorul IN pentru a specifica o lista de valori.

## 7.3. Cautarea prin comparatie (LIKE)

In cadrul clauzei WHERE se poate verifica identitatea dintre doua siruri cu ajutorul operatorilor "=" sau "!="", insa exista cazuri in care este util a se face comparatie intre portiuni ale sirurilor. Comparatia se realizeaza cu ajutorul operatorilor LIKE (asemanator) si NOT LIKE (diferit), iar pentru definirea modelului se folosesc caracterele:

- % - pentru mai multe caractere (orice secventa de caractere)
- \_ - pentru exact un caracter (orice caracter)

## 7.4. Limitarea rezultatelor (LIMIT)

Clauza LIMIT se poate folosi pentru a limita numarul de inregistrari returnat de o instructiune SELECT. In folosirea clauzei LIMIT apar obligatoriu doua argumente: deplasarea si numarul de randuri. Prima valoare indica numarul de ordine al primei inregistrari, iar a doua valoare indica numarul inregistratilor din secventa care urmeaza si care vor fi efectiv vazute de utilizator.

## 7.5. Gruparea randurilor (GROUP BY)

Clauza GROUP BY lista\_coloane grupeaza randuri din setul de rezultate conform coloanelor denumite in lista. Clauza poate fi folosita impreuna cu functii precum COUNT() sau MAX() in clauza *obiecte\_selectate* facandu-se referire folosind nume de coloane sau aliasuri.

## 7.6. Ordonarea rezultatelor (ORDER BY)

Rezultatele interogarilor sunt ordonate asa cum au fost ele introduse in tabele; ele pot fi ordonate folosind clauza ORDER BY a instructiunii SELECT.

Ordinea de sortare se stabileste cu attributele ASC (ascendent) sau DESC (descendent), implicit se foloseste ordonarea ascendenta. Pentru fiecare coloana specificata in clauza de ordonare se poate specifica o ordine proprie de sortare. In cazul in care avem mai multe coloane dupa care se realizeaza ordonarea, in cazul in care inregistrările au valori identice intr-o coloana, acestea vor fi ordonate dupa coloana imediat urmatoare.

## 7.7. Numararea randurilor (COUNT)

Instructiunea SELECT permite determinarea numarului de inregistrari ale unui tabel folosind functia COUNT(). Optional se poate adauga clauza WHERE, in acest caz rezultatul fiind numarul de linii care respecta conditia prezidata in clauza.

De asemenea, clauza GROUP BY permite gruparea inregistratilor returnate de instructiunea SELECT, conform coloanelor precizate in cadrul clauzei.

## 7.8. Selectarea inregistratilor din mai multe tabele (JOIN)

Atunci cand selectarea datelor se face din mai multe tabele in cadrul unei singure instructiuni SELECT, trebuie ca toate tabelele folosite in interogare sa apara in clauza FROM.

Daca sunt specificate mai multe tabele separate prin virgula, instructiunea SELECT returneaza toate combinatiile posibile de randuri din tabele. Utilizarea clauzelor JOIN, CROSS JOIN sau INNER JOIN este similara cu utilizarea virgulelor. Clauza STRAIGHT\_JOIN forteaza unirea tabelor in ordinea in care sunt specificate.

In cazul jonctiunilor, clauza ON inlocuieste clauza WHERE, putandu-se astfel folosi toate expresiile conditionale dintr-o clauza WHERE, inclusiv toti operatorii aritmetici si logici.

Clauza LEFT JOIN forteaza generarea unui rand pentru fiecare rand din tabelul din stanga, chiar daca nu exista nici un echivalent in tabelul din dreapta. Cand nu exista nici o corespondenta, coloanele din tabelul din dreapta sunt returnate sub forma de valori NULL. LEFT OUTER JOIN este echivalenta cu LEFT JOIN.

Jonctiunea la dreapta RIGHT JOIN functioneaza la fel ca LEFT JOIN, doar ca ordinea tabelelor este inversata, deci toate randurile din cel de-al doilea tabel vor fi returnate indiferent daca exista potriviri sau nu cu inregistrari din primul tabel.

NATURAL LEFT JOIN este echivalenta cu LEFT JOIN USING (lista\_coloane), unde lista coloane denumeste fiecare coloana comuna pentru ambele tabele.

## Capitolul 8. INSTRUCȚIUNI DE BAZA

### 8.1. Introducerea înregistrărilor

După crearea bazei de date și a unui tabel inclus în ea, se procedează la introducerea (adaugarea) înregistrărilor (rândurilor) în tabelul respectiv. Există mai multe modalități de a realiza acest lucru:

- ✓ folosind instrucțiunea INSERT;
- ✓ citind înregistrările dintr-un fișier text (folosind instrucțiunea LOAD DATA);
- ✓ citind dintr-un fișier unde se află sub formă de instrucțiuni INSERT.

#### 8.1.1. Introducerea înregistrărilor folosind instrucțiunea INSERT

Instrucțiunea INSERT poate fi utilizată în modul următor:

```
INSERT [LOW_PRIORITY | DELAYED] [IGNORE]
[INTO] nume_tabel (lista_coloane)
VALUES (valori_pentru_coloane)
```

Opțiunea LOW\_PRIORITY amână executia instrucțiunii până în momentul în care nici un utilizator nu mai citește din tabel, iar opțiunea DELAYED pune înregistrarea într-o *coadă de așteptare*, pentru o execuție ulterioară.

Opțiunea IGNORE permite ignorarea înregistrărilor care dublează valorile cheilor unice. Cuvântul cheie INTO poate fi omis dacă se folosesc versiuni recente ale MySQL.

Pentru a avea o sintaxă corectă a instrucțiunii INSERT, trebuie respectate câteva reguli:

- ✓ numerele întregi nu trebuie puse între semne de citare;
- ✓ lista de valori pentru coloane trebuie să conțină valori pentru toate coloanele tabelului, în ordinea în care sunt situate în acesta (dacă acestea nu sunt specificate în mod explicit);
- ✓ tipul valorilor introduse trebuie să coincidă cu tipul coloanelor declarat la crearea tabelului;
- ✓ lista de valori pentru coloane conține siruri în ghilimele;
- ✓ pot fi introduse mai multe înregistrări folosind o singură instrucțiune INSERT, în acest caz valorile pentru coloane cuprinse între paranteze vor fi despartite de „,”.

Adăugarea înregistrărilor numai pentru anumite coloane se face prin enumerarea coloanelor în lista de coloane. Coloanele care nu sunt specificate vor primi valori implicite (prestabilite).

#### 8.1.2. Introducerea înregistrărilor stocate în fișiere text sub formă de date

Pentru utilizarea acestei modalități de introducere a înregistrărilor în tabel, trebuie respectate câteva condiții:

- ✓ fișierul care conține înregistrările trebuie să fie localizat pe calculatorul pe care se află clientul mysql, în același director;

- ✓ datele din fisier trebuie separate de tabulator si trebuie sa aiba aceeasi ordine cu cea a coloanelor din tabel;
- ✓ introducerea mai multor randuri implica separarea acestora printr-un caracter de linie noua (specificat prin propozitia `LINES TERMINATED BY "\r\n"`).

Cu ajutorul comenzii `LOAD DATA` se citesc inregistrari din fisier text si se scriu intr-un tabel. Forma generala a comenzii, este:

```
LOAD DATA [LOW_PRIORITY | CONCURRENT] [LOCAL] INFILE nume_fisier
[REPLACE | IGNORE] INTO TABLE nume_tabel
[FIELDS
  [TERMINATED BY '\t']
  [[OPTIONALLY] ENCLOSED BY '"']
  [ESCAPED BY '\\']
]
[LINES TERMINATED BY '\n']
[IGNORE numar LINES]
[(nume_col1, nume_col2, ...)]
```

Instructiunea citeste inregistrari din fisierul *nume\_fisier* si le scrie in tabelul *nume\_tabel*. Optiunea `LOW_PRIORITY` amana executia instructiunii pana cand nici un utilizator nu mai citeste din tabel.

Daca optiunea `LOCAL` este prezenta, fisierul este cautat pe gazda utilizatorului, iar continutul sau este trimis serverului. Daca optiunea `LOCAL` nu este prezenta, fisierul este cautat pe gazda serverului MySQL.

Optiunea `TERMINATED BY` a propozitiei `FIELDS` permite specificarea delimitatorului dintre valorile situate pe o linie, implicit acest delimitator fiind caracterul de tabulare (`'\t'`). Optiunea `ENCLOSED BY` indica un caracter ce va fi extras (eliminat) de la extremitatile valorilor, atunci cand este gasit.

Propozitia `LINES TERMINATED BY` permite specificarea unui terminator de linie. Implicit acesta este caracterul `'\n'`, dar, in general, poate sa contina mai multe caractere.

Clauza `IGNORE` *numar* `LINES` determina ignorarea primelor *n* linii aflate in fisierul sursa.

### 8.1.3. Introducerea inregistrarilor stocate in fisiere text ca instructiuni `INSERT`

O alta modalitate de introducere a inregistrarilor in tabele se realizeaza prin utilizarea instructiunii `INSERT` stocata intr-un fisier text. La shell-ul sistemului de operare se lanseaza in executie comanda urmatoare:

```
mysql -u utilizator -p parola baza_de_date < fisier_ce_contine_instructiune.txt
```

## 8.2. Actualizarea inregistrarilor - instructiunea `UPDATE`

Pentru modificarea continutului uneia sau a mai multor coloane dintr-o inregistrare existenta este folosita instructiunea UPDATE, cu urmatoarea sintaxa:

```
UPDATE [LOW_PRIORITY] nume_tabel  
SET actualizare  
[WHERE conditie] [LIMIT randuri]
```

unde:

- ✓ nume\_tabel : numele tabelului ale carui inregistrari urmeaza a fi actualizate;
- ✓ actualizare : indica modificarile ce trebuie operate, prezentandu-se sub forma de perechi coloana – valoare (col1 = 'val\_noua\_1', col2 = 'val\_noua\_2');
- ✓ conditie : indica conditia ce trebuie indeplinita pentru ca modificarile sa se rasfranga asupra anumitor inregistrari; in cazul in care lipseste, modificarile vor afecta intregul tabel.

Clauza LIMIT specifica numarul maxim de inregistrari care vor fi actualizate. Clauza LOW\_PRIORITY amana executia instructiunii UPDATE pana cand nici un utilizator nu mai citeste din tabelul care trebuie actualizat.

Clauza WHERE este optionala, fiind folosita pentru a crea instructiuni UPDATE conditionale. Este similara cu folosirea clauzei WHERE in cadrul instructiunilor SELECT, putandu-se folosi toti operatorii de comparatie si cei logici, cum ar fi: =, >, <, OR si AND.

### 8.3. Instructiunea REPLACE

O alta metoda folosita pentru a modifica inregistrari este comanda REPLACE, care se comporta astfel: daca inregistrarea care se doreste a fi inserata in tabel contine o valoare a cheii primare care sa se potriveasca cu o inregistrare deja existenta in tabel, atunci inregistrarea din tabel va fi inlocuita (stearsa), iar noua inregistrare va fi inserata in locul ei. In caz contrar, inregistrarea va fi pur si simplu inserata.

Sintaxa instructiunii REPLACE este urmatoarea:

```
REPLACE INTO nume_tabel (lista_coloane)  
VALUES (valori_pentru_coloane)
```

Asadar, instructiunea REPLACE tine locul secventei de instructiuni DELETE si INSERT.

### 8.4. Stergerea inregistrarilor - instructiunea DELETE

Stergerea inregistrarilor din tabele MySQL se realizeaza prin intermediul instructiunii DELETE, care are urmatoarea sintaxa:

```
DELETE [LOW_PRIORITY] FROM nume_tabel  
[WHERE conditie]  
[LIMIT randuri]
```

Unde :

- ✓ *nume\_tabel* reprezinta numele tabelului din care urmeaza sa se stearga inregistrarile
- ✓ *conditie* reprezinta conditia pe care trebuie sa o indeplineasca inregistrarile din tabel care urmeaza a fi sterse
- ✓ *randuri* specifica numarul maxim de inregistrari care vor fi eliminate

In lipsa clauzei WHERE urmata de conditie, vor fi sterse toate inregistrarile din tabela specificata. Astfel, stergerea conditionala se face prin adaugarea clauzei WHERE si a conditiei ce trebuie indeplinita. Pentru a compune o conditie valida avem la dispozitie operatorii de comparatie si logici.

Clauza LOW\_PRIORITY determina amanarea stergerii pana cand nici un client nu mai citeste din tabel.